Contents

1 Introduction to Programming and C++ **1**
- 1.1 What Is Programming? . . . 1
- 1.2 What Is C++? . . . 1
- 1.3 A Brief History of C++ . . . 1
- 1.4 Why C++ Exists . . . 2
- 1.5 Where C++ Is Used . . . 2
- 1.6 Overview of the Compilation Process . . . 3
- 1.7 Header Files vs. Source Files . . . 3
- 1.8 Translation Units . . . 4
- 1.9 Installing a Compiler and Setting Up the Environment . . . 4
- 1.10 Your First C++ Program . . . 4

2 Fundamental Syntax and Program Structure **6**
- 2.1 The Anatomy of a C++ Program . . . 6
- 2.2 Statements and Expressions . . . 6
- 2.3 Tokens, Keywords, and Identifiers . . . 6
- 2.4 Basic Input and Output . . . 7
- 2.5 Comments and Documentation . . . 7

3 Variables, Types, and Constants **8**
- 3.1 What Are Variables? . . . 8
- 3.2 Declaring and Defining Variables . . . 8
- 3.3 Fundamental Types . . . 8
- 3.4 Literals . . . 8
- 3.5 Uniform Initialization . . . 9
- 3.6 The auto Keyword . . . 9
- 3.7 The decltype Specifier . . . 10
- 3.8 Constants and the const Keyword . . . 10
- 3.9 The constexpr Keyword . . . 10
- 3.10 consteval and constinit (C++20) . . . 11
- 3.11 Type Aliases . . . 11

4 Operators and Expressions **12**
- 4.1 Arithmetic Operators . . . 12
- 4.2 Comparison and Logical Operators . . . 12
- 4.3 Bitwise Operators . . . 12
- 4.4 Assignment and Compound Assignment . . . 12
- 4.5 Increment and Decrement . . . 13
- 4.6 Operator Precedence and Associativity . . . 13
- 4.7 Evaluation Order . . . 13

5 Control Flow **14**
- 5.1 Conditional Statements: if and else . . . 14
- 5.2 The switch Statement . . . 14
- 5.3 Loops: while and do-while . . . 14
- 5.4 The for Loop . . . 15
- 5.5 Jump Statements: break, continue, goto . . . 15

5.6 Scope and Lifetime . . . 15

6 Functions **17**
6.1 What Are Functions? . . . 17
6.2 Function Declarations and Definitions . . . 17
6.3 Parameters vs. Arguments . . . 17
6.4 Default Arguments . . . 17
6.5 Function Overloading . . . 18
6.6 Inline Functions . . . 18
6.7 Recursion . . . 18
6.8 Function Pointers . . . 19
6.9 Introduction to Lambdas . . . 19
6.10 std::function and Callable Objects . . . 19

7 Arrays, Strings, and Dynamic Memory Basics **20**
7.1 Raw Arrays . . . 20
7.2 std::array (C++11) . . . 20
7.3 C-Strings . . . 20
7.4 std::string . . . 21
7.5 std::string_view (C++17) . . . 21

8 Pointers, References, and Memory (Part 1) **22**
8.1 The Memory Model . . . 22
8.2 What Are Pointers? . . . 22
8.3 Pointer Arithmetic . . . 22
8.4 Null Pointers . . . 23
8.5 void Pointers . . . 23
8.6 Dynamic Memory: new and delete . . . 23
8.7 What Are References? . . . 23
8.8 Reference Types in Detail . . . 24
8.9 Pointer vs. Reference . . . 24
8.10 Storage Duration . . . 25
8.11 Alignment . . . 25

9 Object-Oriented Programming: Foundations **26**
9.1 Structs: The Humble Data Aggregate . . . 26
9.2 Classes and Encapsulation . . . 26
9.3 Constructors . . . 26
9.4 Destructors . . . 27
9.5 The this Pointer . . . 27
9.6 const Member Functions . . . 28
9.7 Static Members . . . 28
9.8 Introduction to Inheritance . . . 28

10 Object-Oriented Programming: Advanced **30**
10.1 Access Control Revisited . . . 30
10.2 Virtual Functions and Polymorphism . . . 30
10.3 The Virtual Table (vtable) . . . 30
10.4 Virtual Destructors . . . 31
10.5 Abstract Classes and Interfaces . . . 31
10.6 Copy Semantics . . . 31

10.7 Move Semantics (Introduction) . . . 31
10.8 The Rule of Three, Five, and Zero . . . 32
10.9 Operator Overloading . . . 32
10.10Composition and Aggregation . . . 32

11 Namespaces and Scope Management **34**
11.1 The Problem of Name Collisions . . . 34
11.2 Defining Namespaces . . . 34
11.3 Using Declarations and Directives . . . 34
11.4 Anonymous Namespaces . . . 34
11.5 Inline Namespaces (C++11) . . . 35
11.6 Namespace Aliases . . . 35

12 Type Casting in C++ **36**
12.1 Implicit Conversions . . . 36
12.2 C-Style Casts . . . 36
12.3 static_cast . . . 36
12.4 dynamic_cast . . . 36
12.5 const_cast . . . 37
12.6 reinterpret_cast . . . 37

13 Exception Handling **38**
13.1 The Purpose of Exceptions . . . 38
13.2 Throwing and Catching . . . 38
13.3 Stack Unwinding . . . 38
13.4 noexcept Specifier . . . 38
13.5 Exception Safety Guarantees . . . 39
13.6 std::exception Hierarchy . . . 39

14 Move Semantics in Depth **40**
14.1 The Problem: Unnecessary Copies . . . 40
14.2 lvalues, rvalues, and Value Categories . . . 40
14.3 Rvalue References . . . 40
14.4 std::move . . . 40
14.5 Move Constructor and Move Assignment . . . 41
14.6 Perfect Forwarding . . . 41
14.7 Universal References . . . 42

15 Templates: Generic Programming **43**
15.1 The Problem Templates Solve . . . 43
15.2 Function Templates . . . 43
15.3 Class Templates . . . 43
15.4 Template Specialization . . . 43
15.5 Non-Type Template Parameters . . . 44
15.6 Variadic Templates (C++11) . . . 44
15.7 Fold Expressions (C++17) . . . 44
15.8 Template Metaprogramming Introduction . . . 45
15.9 SFINAE: Substitution Failure Is Not An Error . . . 45
15.10Concepts (C++20) . . . 45
15.11The Curiously Recurring Template Pattern (CRTP) . . . 46

16 The Standard Template Library (STL): Containers **47**
16.1 The Philosophy of the STL . . . 47
16.2 Sequence Containers . . . 47
16.3 Associative Containers . . . 47
16.4 Unordered Associative Containers (C++11) . . . 48
16.5 Container Adaptors . . . 48
16.6 std::array and std::span . . . 48
16.7 Choosing the Right Container . . . 48

17 STL Algorithms and Iterators **49**
17.1 The Iterator Abstraction . . . 49
17.2 Iterator Categories . . . 49
17.3 Non-Mutating Algorithms . . . 49
17.4 Mutating Algorithms . . . 49
17.5 Sorting and Searching . . . 50
17.6 Heap Algorithms . . . 50
17.7 Lambda Expressions and Algorithms . . . 50
17.8 Parallel Algorithms (C++17) . . . 50

18 Advanced Standard Library Types **51**
18.1 std::pair and std::tuple . . . 51
18.2 std::optional (C++17) . . . 51
18.3 std::variant (C++17) . . . 51
18.4 std::any (C++17) . . . 52
18.5 std::span (C++20) . . . 52
18.6 Ranges and Views (C++20) . . . 52
18.7 Chrono Library . . . 52
18.8 Filesystem Library (C++17) . . . 53
18.9 Regular Expressions . . . 53
18.10 Random Number Generation . . . 53
18.11 std::bitset . . . 54

19 Input/Output Streams and File Handling **55**
19.1 The Stream Abstraction . . . 55
19.2 Formatted I/O . . . 55
19.3 String Streams . . . 55
19.4 File Streams . . . 55
19.5 Error Handling in Streams . . . 56
19.6 C++20 Format Library . . . 56

20 Concurrency and Parallelism **57**
20.1 The Threading Model . . . 57
20.2 std::thread . . . 57
20.3 Mutexes and Locks . . . 57
20.4 Deadlock and How to Avoid It . . . 58
20.5 Condition Variables . . . 58
20.6 Futures, Promises, and async . . . 58
20.7 Atomics . . . 59
20.8 Memory Ordering . . . 59

21 Smart Pointers and RAII **60**
21.1 The Tragedy of Raw Pointers 60
21.2 std::unique_ptr 60
21.3 std::shared_ptr 60
21.4 std::weak_ptr 60
21.5 Choosing the Right Smart Pointer 61
21.6 RAII Beyond Memory 61

22 Modern C++ Features and Best Practices **62**
22.1 Modules (C++20) 62
22.2 Undefined Behavior 62
22.3 Performance Considerations 63
22.4 CMake Fundamentals 63
22.5 Static and Shared Libraries 63
22.6 Naming Conventions 64
22.7 Common Interview Mistakes 64
22.8 Debugging and Profiling 64

23 C++23 and Beyond **65**
23.1 Key C++23 Features 65
23.2 The Future of C++ 65

Chapter 1

Introduction to Programming and C++

1.1 What Is Programming?

Programming is the act of instructing a computer to perform specific tasks. A computer, at its lowest level, understands only machine code: sequences of binary digits that represent instructions for the processor. Writing machine code directly is tedious, error-prone, and nearly impossible for large-scale software. Programming languages were invented to bridge the gap between human thought and machine execution.

A programming language provides a structured, human-readable notation for expressing computations. A special program called a compiler or interpreter translates this notation into machine code. Programming involves not just writing code, but understanding the problem, designing a solution, expressing that solution in a language, and verifying correctness. It is a discipline of logic, creativity, and precision.

Why does programming exist? Because software controls nearly every device in modern life: phones, cars, medical equipment, financial systems, and scientific instruments. Programming is the tool that builds these systems. The problem it solves is the need to automate complex, repetitive, or high-speed tasks that humans cannot perform manually. Without programming, computers would be inert silicon and metal.

1.2 What Is C++?

C++ is a general-purpose programming language created by Bjarne Stroustrup at Bell Labs in 1979. It began as "C with Classes," an extension of the C language that added object-oriented features. The name C++ reflects the increment operator ++ in C, symbolizing the language as the next step beyond C.

C++ is a compiled, statically typed language that supports multiple programming paradigms: procedural, object-oriented, generic, and functional. It gives the programmer fine-grained control over system resources, particularly memory, while providing high-level abstractions that simplify complex software design. The language is standardized by the International Organization for Standardization (ISO). The first standard was C++98, followed by C++03, C++11 (a major revolution), C++14, C++17, C++20, and C++23.

Where is C++ used? It powers operating systems, game engines, web browsers, embedded systems, financial trading platforms, databases, and performance-critical scientific simulations. Its combination of zero-overhead abstractions and direct hardware access makes it unique. No other language offers the same level of control with the same breadth of abstraction capabilities.

1.3 A Brief History of C++

Understanding the history of C++ provides context for many of its features and quirks. In the early 1970s, Dennis Ritchie developed C at Bell Labs to write the Unix operating system. C was a small, efficient, procedural language that gave programmers direct memory access through pointers. It became immensely popular for systems programming.

Bjarne Stroustrup, working at Bell Labs in 1979, found that C's procedural model became unwieldy for large projects. He added classes, derived classes, strong type checking, inlining, and default arguments to C, creating "C with Classes" using a preprocessor called Cfront. By 1983, the language had evolved significantly and was renamed C++.

The 1990s saw the rise of the Standard Template Library (STL), developed mainly by Alexander Stepanov, which introduced generic programming through templates. The first ISO standard, C++98, consolidated the core language and the STL. C++03 was a minor bug-fix release. For a decade, the language remained relatively stable, but the programming world moved on: Java, C#, and scripting languages rose in popularity.

The C++11 standard was a watershed moment. It introduced move semantics, lambda expressions, `auto` type deduction, threading support, smart pointers, range-based for loops, and much more. It made C++ feel like a new language. C++14 and C++17 refined these features and added new library components. C++20 delivered concepts, ranges, coroutines, and modules, pushing C++ further toward expressiveness and safety. C++23 continues this evolution.

The evolution from C to modern C++ is a journey from manual, low-level control toward expressive, safe abstractions without sacrificing performance. Modern C++ code can be as concise as Python while running hundreds of times faster, but achieving this requires understanding what happens beneath the surface.

1.4 Why C++ Exists

C++ exists because there was, and still is, a need for a language that combines two seemingly contradictory requirements: direct control over hardware and high-level abstractions for managing complexity. C provides control. Object-oriented languages provide organizational structure. Generic programming provides reusable algorithms. C++ unifies all three.

The core philosophy of C++ is "you only pay for what you use." If you do not use virtual functions, your classes have no vtable overhead. If you use `std::vector`, it is as efficient as a hand-written dynamic array. If you need to manipulate bits directly, you can. If you prefer to work with high-level ranges and views, you can do that as well, and the compiler optimizes away the abstraction layers.

This philosophy makes C++ ideal for resource-constrained environments where garbage collection is unacceptable, for real-time systems where predictability matters, and for massive-scale applications where performance directly translates to cost savings. No garbage collector, no virtual machine, no interpreter stands between the code and the silicon.

1.5 Where C++ Is Used

C++ powers game development. Unreal Engine and many proprietary engines are written in C++ because games require frame budgets of 16 milliseconds or less, leaving no room for garbage collection pauses. C++ runs in every major operating system kernel, including Windows, Linux, and macOS components. Device drivers and firmware are predominantly C and C++.

In finance, high-frequency trading systems use C++ to process market data and execute trades in microseconds. Web browsers like Chrome and Firefox use C++ for their rendering engines. Database systems like MySQL, PostgreSQL, and MongoDB are built in C++. Machine learning frameworks, including TensorFlow and PyTorch, have C++ backends for performance-critical tensor operations. Robotics, aerospace, telecommunications, and scientific computing all rely heavily on C++.

1.6 Overview of the Compilation Process

Compilation transforms human-readable source code into an executable program. The process occurs in several stages. Understanding each stage is essential for diagnosing errors and writing efficient code.

Stage 1: Preprocessing. The preprocessor handles directives beginning with `#`. It expands `#include` by inserting the content of header files, expands macros defined with `#define`, and evaluates conditional compilation with `#ifdef`. The output is a translation unit: a single, self-contained source file with all includes expanded.

Stage 2: Compilation proper. The compiler translates the preprocessed translation unit into assembly language or directly into an object file. It performs lexical analysis (tokenizing), syntax analysis (parsing), semantic analysis (type checking), and optimization. The output is platform-specific assembly code or an object file containing machine code.

Stage 3: Assembly. If the compiler produced assembly, the assembler converts it into a relocatable object file with machine code, data sections, and symbol tables.

Stage 4: Linking. The linker combines one or more object files and libraries into a single executable. It resolves symbols: when one object file references a function defined in another, the linker connects the call to the definition. Static linking copies library code into the executable. Dynamic linking references shared libraries at runtime.

A common mistake for beginners is misunderstanding that `#include` is a textual copy-and-paste operation. It does not magically find function implementations; it only brings in declarations. The linker resolves definitions. Understanding this distinction explains why we have header files and source files.

1.7 Header Files vs. Source Files

C++ separates interface from implementation. A header file (usually `.hpp` or `.h`) contains declarations: function prototypes, class definitions, and variable declarations. A source file (usually `.cpp`) contains definitions: the actual function bodies and static data definitions.

Why separate them? Two reasons. First, compilation speed. If you change a single line in a source file, only that translation unit needs recompilation. Headers are included by many source files, so keeping them minimal and stable reduces build times. Second, organizing interfaces promotes modularity. The header tells a client what a module does; the source file hides how it does it.

A common pattern: the header declares a function.

```
// math_utils.hpp
#pragma once

int add(int a, int b);
double sqrt(double x);
```

The source file defines it.

```
// math_utils.cpp
#include "math_utils.hpp"

int add(int a, int b) {
    return a + b;
}

double sqrt(double x) {
    // Implementation here
```

```
    return 0.0;
}
```

One Definition Rule (ODR). A function or variable can have only one definition across the entire program. Declarations can appear multiple times. This is why headers use include guards (`#pragma once` or `#ifndef...#define`) to prevent duplicate declarations within a single translation unit.

1.8 Translation Units

A translation unit is the fundamental unit of compilation. It consists of a source file plus all headers included by it, after preprocessing. Each translation unit is compiled independently into an object file. The compiler has no knowledge of what exists in other translation units. This independent compilation is why we need header files: they provide the declarations necessary for one translation unit to call functions or use types defined in another.

The linker later connects these independent object files. This architecture allows parallel builds: multiple translation units can be compiled simultaneously on different cores. It also means that changing a function's body in a source file requires only recompiling that one translation unit and relinking, not recompiling the entire project.

1.9 Installing a Compiler and Setting Up the Environment

To write C++, you need a compiler, a text editor or IDE, and optionally a build system. The three major compilers are:

GCC (GNU Compiler Collection) is free, open-source, and the default on Linux. Install via package manager: `sudo apt install g++` on Debian/Ubuntu. **Clang** is part of the LLVM project, known for excellent error messages and fast compilation. Install via `sudo apt install clang`. **MSVC** (Microsoft Visual C++) is part of Visual Studio on Windows. The Community edition is free.

IDEs provide an integrated experience: Visual Studio (Windows), CLion (cross-platform, paid), Qt Creator, and VS Code with extensions. Beginners benefit from an IDE's autocompletion, debugging, and project management. Advanced users often prefer a text editor with a terminal.

Build systems automate the compilation process. Simple projects can use g++ directly:

```
g++ -std=c++20 main.cpp -o program
```

Larger projects need a build system like CMake, which generates platform-specific build files. CMake is the de facto industry standard and will be covered in depth in a later module.

1.10 Your First C++ Program

Every C++ program begins execution in a function called `main`. Here is a minimal program:

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

Line by line: `#include <iostream>` brings in declarations for input and output stream objects. `int main()` defines the entry point. `std::cout` is the standard output stream. The `<<` operator sends data to it. `std::endl` inserts a newline and flushes the output. `return 0;` indicates successful execution to the operating system.

Compile and run:

```
g++ -std=c++20 hello.cpp -o hello
./hello
```

The output appears in the terminal. This simple example already uses the standard library, namespaces, streams, and operator overloading. C++ is dense with meaning even in its simplest form.

Chapter 2

Fundamental Syntax and Program Structure

2.1 The Anatomy of a C++ Program

Every C++ program is a collection of declarations and definitions spread across translation units. The smallest building block is a **token**: a keyword, identifier, literal, operator, or punctuation mark. The compiler reads tokens and groups them into **expressions** and **statements**.

A **statement** performs an action. Most statements end with a semicolon. Compound statements are blocks enclosed in curly braces. Declarations introduce names; definitions provide the entities those names refer to.

```
int x;              // declaration and definition
extern int y;       // declaration only, no definition
x = 5;              // expression statement
{                   // compound statement (block)
    int z = x + 1;
}                   // z destroyed here
```

The `main` function is special: it is the entry point called by the C++ runtime. It must return `int`. A return of `0` or reaching the end of `main` without a return both indicate success.

2.2 Statements and Expressions

An **expression** computes a value. It may consist of literals, variables, operators, and function calls. Every expression has a type and may have side effects. Examples:

```
5 + 3               // evaluates to int 8
x * y + getVal()    // calls function, multiplies, adds
a = b               // assignment; evaluates to a (lvalue)
```

An expression followed by a semicolon becomes an expression statement. Declarations, control flow constructs, and jump statements are other kinds of statements. The null statement is a lone semicolon, valid but usually a mistake.

C++ distinguishes between **lvalues** and **rvalues**. An lvalue refers to an object with an identifiable address. An rvalue is a temporary or literal that does not persist beyond the expression. These concepts become critical when we discuss move semantics later.

2.3 Tokens, Keywords, and Identifiers

A **token** is the smallest meaningful unit in C++ source code. Types of tokens: keywords (`int`, `return`, `if`), identifiers (user-chosen names), literals (`42`, `3.14`, `'c'`), operators (`+`, `«`, `&&`), and punctuators (`;`, `(`, `}`).

Keywords are reserved words with fixed meaning. You cannot use them as identifiers. C++ has approximately 90 keywords, including `auto`, `break`, `case`, `class`, `const`, `constexpr`,

`default`, `delete`, `enum`, `explicit`, `extern`, `friend`, `inline`, `namespace`, `noexcept`, `nullptr`, `override`, `template`, `throw`, `try`, `using`, `virtual`, and many more.

Identifiers name entities: variables, functions, classes, namespaces. Identifiers consist of letters, digits, and underscores; they cannot start with a digit. Case matters: `myVar` and `myvar` are distinct. Underscore-prefixed names and double-underscore names are reserved for the implementation; avoid them.

2.4 Basic Input and Output

The `<iostream>` header provides `std::cin` (standard input), `std::cout` (standard output), and `std::cerr` (standard error). These are objects of types `std::istream` and `std::ostream`. The « operator inserts formatted data into an output stream. The » operator extracts formatted data from an input stream.

```
#include <iostream>
#include <string>

int main() {
    std::string name;
    int age;

    std::cout << "Enter your name: ";
    std::cin >> name;
    std::cout << "Enter your age: ";
    std::cin >> age;

    std::cout << "Hello, " << name
              << ". You are " << age << " years old.\n";
    return 0;
}
```

Input extraction stops at whitespace by default. For full-line input, use `std::getline`. Note that
`n` is a newline character; `std::endl` also flushes the buffer, which can be costly if used excessively.

2.5 Comments and Documentation

Comments explain intent. C++ supports two styles: single-line comments starting with `//`, and multi-line comments delimited by `/*` and `*/`. Multi-line comments do not nest; nesting causes compile errors.

```
// This is a single-line comment

/*
 * This is a multi-line comment.
 * It spans multiple lines.
 */
```

Use comments to explain *why* the code does something, not *what* it does. The code itself should be clear enough to show what happens. For documenting interfaces, tools like Doxygen extract specially formatted comments to generate API documentation.

Chapter 3

Variables, Types, and Constants

3.1 What Are Variables?

A variable is a named storage location in memory that holds a value of a specific type. Variables let programs store, manipulate, and retrieve data. Before C++17, a variable had three properties: a name, a type, and a value. The type determines the size in memory, the range of possible values, and the set of valid operations.

Why variables exist: without them, programs could only compute immediate values. Variables enable stateful computation where results build upon previous results. They are fundamental to all imperative programming.

3.2 Declaring and Defining Variables

A declaration introduces a name and its type into the current scope. A definition also allocates storage. Most variable declarations are also definitions:

```
int count;          // definition: allocates sizeof(int) bytes
double price = 9.99; // definition with initialization
extern int global; // declaration only; definition elsewhere
```

Every variable must be defined exactly once. It may be declared many times. Failure to define a used variable results in a linker error. Defining a variable more than once violates the One Definition Rule.

3.3 Fundamental Types

C++ provides a set of built-in **fundamental types**:

Boolean: `bool` holds either `true` or `false`. Size is typically 1 byte, but this is implementation-defined.

Character types: `char` stores a single character, typically 1 byte. `wchar_t` for wide characters, `char8_t` (C++20) for UTF-8, `char16_t` for UTF-16, `char32_t` for UTF-32.

Integer types: `short`, `int`, `long`, `long long`, each available in `signed` and `unsigned` forms. Their sizes are platform-dependent; the standard guarantees minimum ranges. Use `int` for most integers. Use `unsigned` only for bit manipulation or when negative values are meaningless.

Floating-point types: `float` (single precision), `double` (double precision), `long double` (extended precision). `double` is the default for floating-point computation.

Void: `void` represents the absence of a type. Used for functions that return nothing, and as a generic pointer type (`void*`).

For precise-sized integers, include `<cstdint>` and use `std::int32_t`, `std::uint64_t`, and similar aliases. This is essential for portable code that depends on specific bit widths.

3.4 Literals

Literals represent constant values embedded in source code. Their syntax determines their type:

```
42          // int
42L         // long
42LL        // long long
42u         // unsigned int
42ul        // unsigned long
3.14        // double
3.14f       // float
3.14L       // long double
'a'         // char
"hello"     // const char[6] (null-terminated)
u8"hi"      // UTF-8 string literal (C++20 char8_t)
true        // bool
false       // bool
nullptr     // std::nullptr_t
```

Integer literals can be octal (`012`), decimal (`10`), hexadecimal (`0xA`), or binary (`0b1010` since C++14). Digit separators (`'` since C++14) improve readability: `1'000'000`.

3.5 Uniform Initialization

C++ offers several initialization syntaxes. The traditional form uses an equals sign. Direct initialization uses parentheses. Since C++11, **uniform initialization** uses curly braces:

```
int a = 5;      // copy initialization
int b(5);       // direct initialization
int c{5};       // uniform (brace) initialization
int d = {5};    // copy list initialization
int e{};        // value initialization, e is 0
```

Uniform initialization is preferred in modern C++ for several reasons. It prevents **narrowing conversions**, where a value would lose information:

```
int x = 3.14;   // compiles with warning, x = 3
int y{3.14};    // compile error: narrowing conversion
```

It also avoids the **most vexing parse**, where a declaration looks like a function declaration:

```
Widget w();     // function declaration, not object!
Widget w{};     // value-initialized Widget object
```

However, brace initialization interacts with `std::initializer_list` constructors. When a class has an `initializer_list` constructor, braces prefer it over other constructors. This can surprise programmers. Understanding these subtleties is part of mastering C++.

3.6 The auto Keyword

The `auto` keyword tells the compiler to deduce a variable's type from its initializer. It was repurposed in C++11 (previously a storage class specifier).

```
auto i = 42;            // int
auto d = 3.14;          // double
auto s = "hello";       // const char*
auto v = std::vector<int>{1, 2, 3}; // std::vector<int>
```

Use `auto` when the type is obvious from context, when it is long or unutterable (like lambda closures or iterator types), or in generic code. Avoid `auto` when explicitly stating the type improves readability. Never use bare `auto` when you need a reference or const qualification; the deduction strips references and top-level cv-qualifiers:

```
const int& getRef();
auto x = getRef();   // int, not const int&
decltype(auto) y = getRef(); // const int& (C++14)
const auto& z = getRef();    // const int&
```

3.7 The decltype Specifier

`decltype` yields the declared type of an expression without evaluating it. It is essential in generic programming:

```
int x = 5;
decltype(x) y = 10;  // y is int

template<typename T, typename U>
auto add(T t, U u) -> decltype(t + u) {
    return t + u;
}
```

The trailing return type syntax in the example allows the return type to depend on parameter types. C++14 allows `decltype(auto)` for perfect forwarding of return types.

3.8 Constants and the const Keyword

The `const` keyword makes a variable read-only. Once initialized, its value cannot change. This communicates intent to the compiler and to other programmers. The compiler can enforce const correctness, catching accidental modifications.

```
const int max_connections = 100;
max_connections = 200;  // error: assignment to const

void print(const std::string& s) {
    // s cannot be modified here
    std::cout << s;
}
```

const with pointers requires care. Read right to left:

```
const int* p;       // pointer to const int (data can't change)
int* const p;       // const pointer to int (pointer can't change)
const int* const p; // const pointer to const int
```

Top-level const (constness of the object itself) is ignored when passing by value. `const int` and `int` are the same type in function parameter contexts. This can be confusing but is consistent with the fact that the caller's argument is copied anyway.

3.9 The constexpr Keyword

`constexpr` declares that a variable or function's value is computable at compile time. This enables compile-time evaluation, reducing runtime overhead and allowing values to be used in contexts requiring constants (array sizes, template arguments).

```
constexpr int square(int x) { return x * x; }
constexpr int val = square(5); // computed at compile time
int arr[val];                  // OK, val is a constant expression
```

A constexpr function is not guaranteed to execute at compile time. It can also be called with runtime arguments. The compiler evaluates it at compile time only when all arguments are constant expressions and the result is used in a constant context. C++14 relaxed restrictions on constexpr functions, allowing multiple statements, conditionals, and loops. C++20 extended this further with consteval and constinit.

3.10 consteval and constinit (C++20)

consteval creates an **immediate function**. Every call to a consteval function must produce a compile-time constant. Unlike constexpr, there is no runtime fallback.

```
consteval int compile_square(int x) { return x * x; }
int a = compile_square(5);  // OK, constant expression
int b = 5;
int c = compile_square(b);  // error: b not constant
```

constinit ensures that a variable with static storage duration is initialized at compile time. It prevents the static initialization order fiasco without requiring constexpr.

```
constinit int global = 42; // OK, initialized at compile time
int runtime_val = compute();
constinit int bad = runtime_val; // error: not constant
```

3.11 Type Aliases

Type aliases give alternative names to existing types, improving readability and reducing typing. Two syntaxes exist:

```
typedef unsigned long ulong;          // traditional
using ulong = unsigned long;          // modern (C++11)
using callback = void(*)(int);        // function pointer alias
template<typename T>
using vec = std::vector<T>;           // alias template
```

Prefer the using syntax. It is more readable, especially for complex function pointer types, and it supports template aliases which typedef cannot.

Chapter 4

Operators and Expressions

4.1 Arithmetic Operators

C++ provides the standard arithmetic operators: addition (+), subtraction (-), multiplication (*), division (/), and modulus (%). Division of integers truncates toward zero. Modulus works only on integral operands. Unary plus and minus are also available.

Integer overflow for signed types is undefined behavior. Unsigned integers wrap around modulo. This is a notorious source of bugs. Use checked arithmetic or the functions in `<numeric>` for safe operations.

```
int a = 7 / 2;     // 3, not 3.5
int b = 7 % 2;     // 1
unsigned u = 0u;
u = u - 1;         // wraps to UINT_MAX
```

4.2 Comparison and Logical Operators

Comparison operators (==, !=, <, >, <=, >=) yield `bool`. Logical operators (!, &&, ||) work on `bool` operands. Both && and || short-circuit: the second operand is evaluated only if necessary.

```
if (p != nullptr && p->value > 0) { ... } // safe: short-circuit
```

A common mistake is using = (assignment) when == (comparison) is intended. Compilers warn about this if the assignment is used as a condition, but explicit comparison is clearer.

4.3 Bitwise Operators

Bitwise operators manipulate individual bits: ~ (NOT), & (AND), | (OR), ^ (XOR), « (left shift), » (right shift). They are essential for low-level programming, flags, and performance optimizations.

Right shift of a signed negative integer is implementation-defined. Prefer unsigned integers for bit manipulation.

```
unsigned flags = 0b1010;
flags |= 0b0100;     // set bit 2
flags &= ~0b1000;    // clear bit 3
bool is_set = flags & 0b0010; // test bit 1
```

4.4 Assignment and Compound Assignment

Assignment (=) stores a value in a variable and returns an lvalue referring to the left operand, enabling chaining: `a = b = c = 0`. Compound assignment operators (+=, -=, *=, /=, %=, «=, »=, &=, |=, ^=) combine an operation with assignment.

```
x += 5;   // equivalent to x = x + 5
```

4.5 Increment and Decrement

The prefix increment (++x) increments and returns the new value as an lvalue. The postfix increment (x++) increments but returns a copy of the old value as an rvalue. Prefer prefix forms unless the old value is specifically needed. For user-defined iterators, postfix creates an unnecessary temporary that may be expensive.

4.6 Operator Precedence and Associativity

Operator precedence determines the order in which operators are applied in expressions with multiple operators. Associativity determines grouping when operators have the same precedence.

For example, * has higher precedence than +, so `3 + 4 * 5` is 23, not 35. Assignment is right-associative, so `a = b = 0` assigns zero to `b` and then to `a`.

Memorizing the entire precedence table is unnecessary. Use parentheses to make intent explicit. Code is read far more often than it is written, and clarity trumps brevity.

4.7 Evaluation Order

The order in which subexpressions are evaluated is largely unspecified in C++. This can cause subtle bugs when expressions have side effects:

```
int i = 0;
std::cout << i << " " << i++; // unspecified: 0 0 or 1 0?
```

Avoid expressions where a variable is both read and modified without an intervening sequence point. C++17 introduced some guarantees: the left operand of « is evaluated before the right operand, and in `a = b`, `b` is evaluated before `a` (but not their subexpressions). Still, write clear, simple expressions.

Chapter 5

Control Flow

5.1 Conditional Statements: if and else

The `if` statement executes code conditionally:

```
if (condition) {
    // executed when condition is true
} else if (other_condition) {
    // executed when first false and second true
} else {
    // executed when all conditions false
}
```

The condition must be convertible to `bool`. C++17 introduced **if with initializer**:

```
if (auto it = map.find(key); it != map.end()) {
    use(it->second);
} // it goes out of scope here
```

This restricts the scope of the variable to the if-else block, preventing accidental use later.

5.2 The switch Statement

`switch` selects among multiple alternatives based on an integral or enumeration value:

```
switch (value) {
    case 1:
        handle_one();
        break;
    case 2:
        handle_two();
        break;
    default:
        handle_other();
}
```

Fallthrough occurs if `break` is omitted; execution continues into the next case. This is occasionally intentional (with a comment like `[[fallthrough]]` in C++17) but usually a bug. Always end non-empty cases with `break` or `return`. The `default` case catches all unmatched values.

C++17 allows an initializer in `switch`, just like `if`.

5.3 Loops: while and do-while

The `while` loop repeats a block while a condition is true. The condition is checked before each iteration; zero iterations are possible.

```
while (condition) {
    // body
}
```

The `do-while` loop guarantees at least one iteration because the condition is checked after the body:

```
do {
    // body
} while (condition);
```

5.4 The for Loop

The traditional `for` loop has three parts: initialization, condition, and increment.

```
for (int i = 0; i < 10; ++i) {
    std::cout << i;
}
```

The range-based `for` loop (C++11) simplifies iteration over containers:

```
std::vector<int> v = {1, 2, 3};
for (int x : v) {
    std::cout << x;
}
for (auto& x : v) {  // reference to modify
    x *= 2;
}
for (const auto& x : v) { // const reference for read-only
    std::cout << x;
}
```

Under the hood, the range-based for calls `begin()` and `end()` on the container. It works with any type that provides these, including C arrays of known length.

5.5 Jump Statements: break, continue, goto

`break` exits the innermost loop or switch. `continue` skips the rest of the current loop iteration and proceeds to the next condition check.

`goto` jumps to a labeled statement within the same function. It is rarely used in modern C++ and is generally discouraged because it can create spaghetti code. Valid use cases include breaking out of deeply nested loops or complex state machines where alternatives are even less clear. Never jump over variable initialization.

```
for (int i = 0; i < 100; ++i) {
    if (i == 50) break;       // exit loop
    if (i % 2 == 0) continue; // skip evens
    process(i);
}
```

5.6 Scope and Lifetime

Scope is the region of program text where a name is visible. **Lifetime** is the runtime period during which an object exists. C++ uses block scope: a variable declared in a block `{}` is visible from its declaration to the end of the block.

```
{
    int x = 5; // x constructed here
    {
        int y = 10; // nested scope
    } // y destroyed here
} // x destroyed here
```

Objects are destroyed in reverse order of construction. This deterministic destruction is a cornerstone of C++'s resource management model (RAII).

Chapter 6

Functions

6.1 What Are Functions?

A function is a named block of reusable code that performs a specific task. Functions serve multiple purposes: they break complex problems into manageable pieces, eliminate code duplication, enable code reuse across projects, and provide abstraction boundaries.

A function has a name, a return type, a parameter list, and a body. The signature of a function consists of the name and the parameter types (not the return type). The compiler uses signatures to distinguish overloads.

6.2 Function Declarations and Definitions

A **declaration** (prototype) announces the function's existence and signature without providing the body. A **definition** provides the body. You can declare a function many times but must define it exactly once.

```
// declaration (prototype)
int max(int a, int b);

// definition
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Place declarations in headers and definitions in source files, unless the function is a template or inline, in which case the definition typically goes in the header.

6.3 Parameters vs. Arguments

Parameters are the variables listed in the function definition. **Arguments** are the values passed at the call site. C++ supports **pass by value** (a copy is made), **pass by pointer** (an address is copied), and **pass by reference** (an alias is created, no copy).

```
void by_value(int x);        // x is a copy
void by_pointer(int* x);     // x points to original
void by_reference(int& x);   // x is the original
```

Modern C++ prefers pass by const reference for large objects to avoid copying while preventing modification:

```
void print(const std::string& s);
```

6.4 Default Arguments

Default arguments allow callers to omit parameters, which take a predefined value:

```
void log(std::string\_view msg, int level = 0) {
    // ...
}

log("Error");       // level is 0
log("Warning", 1); // level is 1
```

Default arguments are specified in the declaration (usually the header) and not repeated in the definition. A parameter with a default must have all following parameters also defaulted.

6.5 Function Overloading

Overloading allows multiple functions with the same name but different parameter lists. The compiler selects the best match based on argument types at the call site.

```
void print(int x);
void print(double x);
void print(const std::string& s);
```

Overload resolution is complex. It considers exact matches, promotions, conversions, and ellipsis in order of preference. Ambiguous calls cause compile errors. Avoid overloading on types that are closely related or on integer types that may cause unexpected promotions.

6.6 Inline Functions

The `inline` keyword hints that the compiler should replace function calls with the function body, eliminating call overhead. The compiler is free to ignore this hint. In modern C++, `inline` primarily serves to permit multiple definitions across translation units (ODR waiver). Inline functions and function templates are typically defined in headers.

```
inline int square(int x) {
    return x * x;
}
```

Member functions defined inside a class definition are implicitly inline. Overusing inline can bloat binary size. Trust the compiler's optimization heuristics unless profiling indicates a problem.

6.7 Recursion

A function that calls itself is recursive. Recursion is a natural way to express algorithms on recursive data structures like trees.

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Every recursive function needs a base case to terminate. Deep recursion can overflow the call stack. Some recursive algorithms can be transformed into iterative ones using an explicit stack or tail-call optimization, which C++ compilers may or may not perform.

6.8 Function Pointers

A function pointer stores the address of a function, enabling callbacks and runtime dispatch. The syntax is archaic but still used.

```
void greet(const std::string& name) {
    std::cout << "Hello, " << name;
}

void (*func_ptr)(const std::string&) = &greet;
func_ptr("World");
```

For type aliases, `using callback = void(*)(const std::string&);` is much cleaner. Function pointers cannot capture state. For stateful callbacks, use function objects or lambdas.

6.9 Introduction to Lambdas

A lambda expression creates an anonymous function object (closure) inline. C++11 introduced lambdas with the syntax:

```
[capture_list](parameters) -> return_type { body }
```

```
auto add = [](int a, int b) -> int { return a + b; };
int result = add(3, 4);
```

The return type can be omitted if the body consists of a single return statement or is void. The capture list specifies which variables from the enclosing scope are accessible inside the lambda:

```
int multiplier = 2;
auto times = [multiplier](int x) { return x * multiplier; };
// multiplier is captured by value (copy)

auto increment = [&counter](int x) { counter += x; };
// counter is captured by reference
```

Capture defaults are `[=]` (capture everything by value) and `[&]` (capture everything by reference). Prefer explicit captures to avoid accidental dangling references or copies. Lambdas are syntactic sugar for function objects; we will explore their implementation later.

6.10 std::function and Callable Objects

`std::function` (from `<functional>`) is a polymorphic wrapper for any callable target: functions, function pointers, lambdas, or function objects. It uses type erasure to store targets of any compatible signature.

```
#include <functional>
std::function<int(int, int)> op;
op = [](int a, int b) { return a + b; };
op = std::multiplies<int>{}; // standard library functor
int result = op(5, 3);
```

`std::function` incurs some overhead compared to templates. Use it when you need to store or pass heterogeneous callables at runtime. Prefer templates when the type is known at compile time.

Chapter 7

Arrays, Strings, and Dynamic Memory Basics

7.1 Raw Arrays

A C-style array is a contiguous block of elements of the same type, allocated on the stack or statically.

```
int arr[5];                    // uninitialized
int arr2[5] = {1, 2, 3};      // remaining elements zero-initialized
int arr3[] = {1, 2, 3, 4};    // size deduced as 4
```

Arrays decay to pointers when passed to functions or in most expression contexts. The size information is lost.

```
void func(int* arr);  // no size information
```

This decay is a major source of bugs and a primary reason `std::array` and `std::vector` are preferred in modern code. Raw arrays have no bounds checking, no assignment, and no size query.

7.2 std::array (C++11)

`std::array` is a zero-overhead wrapper around a C-style array. It preserves size, supports standard container interfaces, and does not decay to a pointer.

```
#include <array>
std::array<int, 5> arr = {1, 2, 3, 4, 5};
for (const auto& elem : arr) { std::cout << elem; }
std::cout << arr.size(); // 5
std::array<int, 5> arr2 = arr; // copyable!
```

`std::array` is a compile-time sized container. Its size must be a constant expression. For runtime-sized arrays, use `std::vector`.

7.3 C-Strings

A C-string is a null-terminated array of `char`. The null character (’ `textbackslash 0’`) marks the end. Standard library functions in `<cstring>` manipulate them: `strlen`, `strcpy`, `strcat`, `strcmp`.

```
const char* s = "Hello"; // 6 characters: Hello + null
char buffer[20];
strcpy(buffer, s); // dangerous: no bounds checking
```

C-strings are error-prone: buffer overflows, off-by-one errors for the null terminator, and manual memory management. Prefer `std::string` for text.

7.4 std::string

`std::string` is a dynamic, resizable string class that manages its own memory. It supports concatenation, comparison, substring extraction, and seamless interoperability with C-strings.

```
std::string s = "Hello";
s += " World";
std::cout << s.length();  // 11
std::string sub = s.substr(0, 5); // "Hello"
if (s == "Hello World") { ... }
const char* cstr = s.c_str(); // get C-string when needed
```

Small String Optimization (SSO) means short strings are stored within the string object itself, avoiding heap allocation. `std::string` is a workhorse class; study its interface thoroughly.

7.5 std::string_view (C++17)

`std::string_view` is a non-owning view of a character sequence. It can point to a `std::string`, a C-string, or a substring, without copying.

```
void print(std::string\_view sv) {
    std::cout << sv;
}

std::string s = "Hello World";
print(s);                   // works
print("Literal");           // works
print(s.substr(0, 5));      // no copy!
```

`string_view` does not own the data. It is the caller's responsibility to ensure the viewed data outlives the view. Never return a `string_view` to a local string. This is a common dangling reference bug.

Chapter 8

Pointers, References, and Memory (Part 1)

8.1 The Memory Model

A computer's memory is a linear array of bytes, each with a unique address. The operating system and C++ runtime organize this memory into distinct regions:

The stack stores local variables and function call frames. It grows and shrinks automatically as functions are called and return. Allocation is fast (a single CPU instruction moving the stack pointer). Stack space is limited (typically a few megabytes).

The heap stores dynamically allocated objects with lifetime controlled by the programmer. Allocation requires a system call or library function and is slower. Heap space is much larger, bounded by available system memory.

Static memory stores global and static variables. They are allocated when the program starts and deallocated when it ends.

Text stores the compiled machine code. It is usually read-only.

Understanding these regions is fundamental because pointers can refer to any of them, but the semantics of lifetime and ownership differ dramatically.

8.2 What Are Pointers?

A pointer is a variable that stores a memory address. It "points to" an object of a specific type. A pointer's type tells the compiler how to interpret the bits at that address.

```
int x = 42;
int* p = &x;  // p stores the address of x
*p = 10;      // dereference: access the pointed-to int
```

The operator `&` takes the address of an object. The operator `*` dereferences a pointer, accessing the object it points to. Pointers are themselves objects and can be stored, copied, reassigned, and compared.

Why pointers exist: they enable dynamic memory allocation, pass-by-reference semantics (in C), runtime polymorphism, data structure building (linked lists, trees), and efficient parameter passing for large objects. They are one of C++'s most powerful and dangerous features.

8.3 Pointer Arithmetic

Adding an integer to a pointer advances it by that many elements, not bytes. The compiler scales the offset by the size of the pointed-to type.

```
int arr[5] = {10, 20, 30, 40, 50};
int* p = arr;      // points to arr[0]
p = p + 2;         // points to arr[2]
std::cout << *p;   // 30
```

Pointer arithmetic is valid only within the bounds of an array (plus one past the end). Accessing outside these bounds is undefined behavior. Subtracting two pointers gives the number of elements between them.

8.4 Null Pointers

A null pointer points to nothing. Before C++11, programmers used `NULL` (macro) or `0`. C++11 introduced `nullptr`, a keyword of type `std::nullptr_t`.

```
int* p = nullptr;
if (p) { ... }          // false
if (p == nullptr) { ... } // true
```

Always initialize pointers to `nullptr` if they do not yet have a valid address. Dereferencing a null pointer is undefined behavior and typically crashes the program. Always check for null before dereferencing if the pointer might be null.

8.5 void Pointers

A `void*` is a generic pointer that can hold the address of any object type. It cannot be dereferenced directly because the compiler does not know the object's size. It must be cast to a specific type first.

```
int x = 5;
void* v = &x;
int* p = static\_cast<int*>(v); // cast back to int*
std::cout << *p;
```

`void*` is used in C interfaces and type-erasure patterns. In modern C++, prefer templates or `std::any` for type safety.

8.6 Dynamic Memory: new and delete

`new` allocates memory on the heap for an object, calls its constructor, and returns a pointer to it. `delete` calls the destructor and deallocates the memory.

```
int* p = new int(42);   // allocate and initialize
std::cout << *p;
delete p;               // deallocate

int* arr = new int[100]; // array allocation
delete[] arr;            // array deallocation
```

Every `new` must have a matching `delete`. Every `new[]` must have a matching `delete[]`. Mismatching these or forgetting to delete causes memory leaks or undefined behavior. Manual memory management is the most common source of bugs in C and old-style C++. Modern C++ uses smart pointers and containers to automate this, a topic for the next chapter.

8.7 What Are References?

A reference is an alias for an existing object. Once initialized, it cannot be reseated to refer to a different object. References are not objects themselves (though in some contexts the compiler synthesizes pointer behavior behind the scenes).

```
int x = 10;
int& ref = x;    // ref is an alias for x
ref = 20;        // x is now 20
int y = 30;
ref = y;         // assigns 30 to x, does NOT rebind ref
```

References provide pass-by-reference without pointer syntax. They are safer than pointers because they cannot be null (well-formed references must be initialized) and cannot be reseated.

8.8 Reference Types in Detail

lvalue references (declared with `&`) bind to lvalues. They are the classic reference type.

const references (declared with `const T&`) can bind to both lvalues and rvalues, and extend the lifetime of temporaries:

```
const std::string& s = getString(); // lifetime extended
```

rvalue references (declared with `T&&`) bind to rvalues and enable move semantics. They will be covered extensively in the advanced topics chapters.

Universal (forwarding) references occur when `T&&` appears with type deduction (`auto&&` or template parameter). They can bind to anything and preserve value category. This is also covered later.

8.9 Pointer vs. Reference

When to use a pointer:

- The referred object may not exist (null is possible).
- The reference needs to be reassigned to a different object.
- Ownership transfer is involved (smart pointers).
- Interfacing with C APIs.

When to use a reference:

- The referred object must exist.
- No reassignment is needed.
- Operator overloading (references enable natural syntax).
- Copy constructors and assignment operators.

Prefer references for function parameters when the object is guaranteed to exist and you do not transfer ownership. Use pointers when nullability or reseating is required.

8.10 Storage Duration

Every C++ object has a **storage duration** that governs its lifetime.

Automatic storage duration: local variables declared without `static`, `extern`, or `thread_local`. Destroyed at end of block.

Static storage duration: global variables, static local variables, static members. Initialized once (often at program start) and destroyed at program exit.

Thread-local storage duration (C++11): each thread has its own instance of the variable. Declared with `thread_local`.

Dynamic storage duration: created with `new` and destroyed with `delete`. Lifetime is programmer-managed.

```
int global;             // static
void func() {
    static int calls = 0; // static: persists across calls
    int local;             // automatic
    calls++;
}
```

8.11 Alignment

Objects in memory must be stored at addresses divisible by their alignment requirement. Alignment improves performance because CPUs fetch memory in aligned chunks. `alignof(T)` queries the alignment of a type. `alignas(N)` specifies a stricter alignment for a variable or type.

```
alignas(64) int cache_line_var; // aligned to 64 bytes
std::cout << alignof(int);       // typically 4
```

Misaligned access on some architectures causes hardware exceptions; on others, it incurs performance penalties. The compiler handles alignment automatically in most cases. Manual alignment is needed for SIMD, cache optimization, and interfacing with hardware.

Chapter 9

Object-Oriented Programming: Foundations

9.1 Structs: The Humble Data Aggregate

A `struct` in C++ is a user-defined type that groups related data members. In C, structs are the primary way to create compound types. C++ structs are classes with default public access, but they are typically used for plain data aggregates.

```
struct Point {
    double x;
    double y;
};

Point p{3.0, 4.0};
std::cout << p.x << ", " << p.y;
```

Use structs when the type is a simple bundle of data with little or no invariant, and no need for encapsulation. If you start adding constructors, destructors, or private members, consider using a class.

9.2 Classes and Encapsulation

A `class` is identical to a struct except members are private by default. Classes are the primary mechanism for implementing objects with behavior and invariants. **Encapsulation** hides implementation details behind a public interface.

```
class BankAccount {
private:
    double balance;
public:
    BankAccount() : balance(0.0) {}
    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    double getBalance() const { return balance; }
};
```

The `private` members are accessible only from within the class. `public` members form the interface. `protected` members are accessible to derived classes (discussed later). Encapsulation prevents accidental modification of invariants, reduces coupling, and allows implementation changes without affecting clients.

9.3 Constructors

A constructor initializes an object at creation. It has the same name as the class and no return type. Multiple constructors can be overloaded.

```
class Widget {
    int id;
    std::string name;
public:
    Widget() : id(0), name("Unnamed") {}  // default constructor
    Widget(int i) : id(i), name("") {}     // converting constructor
    Widget(int i, std::string n) : id(i), name(std::move(n)) {}
};
```

The **member initializer list** (`: id(0), name("Unnamed")`) initializes members before the constructor body executes. This is more efficient than assignment in the body, and it is required for const members, reference members, and objects without default constructors. Always prefer the initializer list.

If no constructor is declared, the compiler generates a default constructor. If any constructor is declared, the generated one is suppressed unless explicitly defaulted with `= default`.

9.4 Destructors

A destructor cleans up when an object's lifetime ends. It has the same name as the class prefixed with , takes no parameters, and returns nothing.

```
class FileHandle {
    FILE* file;
public:
    FileHandle(const char* path) {
        file = fopen(path, "r");
    }
    ~FileHandle() {
        if (file) fclose(file);
    }
};
```

Destructors are called automatically when automatic objects go out of scope, when dynamically allocated objects are deleted, and when temporaries are destroyed. This deterministic destructor invocation is the foundation of RAII (Resource Acquisition Is Initialization). The compiler generates a destructor if none is declared.

9.5 The this Pointer

Inside non-static member functions, `this` is a pointer to the object on which the function was called. Its type is `T* const` (in non-const methods) or `const T* const` (in const methods).

```
class Counter {
    int value;
public:
    Counter& increment() {
        ++value;
        return *this; // enables method chaining
    }
};
```

`this` is rarely written explicitly except to disambiguate member names from parameters or to enable chaining.

9.6 const Member Functions

A member function declared `const` promises not to modify the object. The compiler enforces this: only const functions can be called on const objects or through const references.

```
class Foo {
    int x;
public:
    int getX() const { return x; } // read-only
    void setX(int val) { x = val; }
};

void print(const Foo& f) {
    f.getX();  // OK
    f.setX(5); // error: setX is non-const
}
```

Const correctness is a hallmark of well-designed C++ code. It catches accidental modifications at compile time and documents intent.

9.7 Static Members

Static data members belong to the class, not individual objects. They are shared by all instances. Static member functions can access only static data; they have no `this` pointer.

```
class Registry {
    static int count;
public:
    static int getCount() { return count; }
};
int Registry::count = 0; // definition in source file
```

Since C++17, static data members can be declared `inline` in the class definition, eliminating the need for an out-of-line definition:

```
class Registry {
    inline static int count = 0; // C++17
};
```

9.8 Introduction to Inheritance

Inheritance creates a hierarchical relationship where a derived class inherits members from a base class. It models "is-a" relationships.

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override { return 3.14159 * radius * radius;
       }
```

```
};
```

`public` inheritance means the derived class is a subtype of the base. `protected` and `private` inheritance are for implementation details (composition is usually better). Virtual functions enable runtime polymorphism. Pure virtual functions (`= 0`) make a class abstract: it cannot be instantiated directly.

Note: Inheritance is a deep topic. The following modules expand on polymorphic behavior, virtual tables, slicing, and the proper use of inheritance versus composition.

Chapter 10

Object-Oriented Programming: Advanced

10.1 Access Control Revisited

Access specifiers govern visibility of base class members in derived classes:

public inheritance: public base members remain public, protected remain protected. This is the standard "is-a" relationship.

protected inheritance: public and protected base members become protected in the derived class. Rarely used; composition is usually a better choice.

private inheritance: public and protected base members become private. This implements "is-implemented-in-terms-of" without public interface exposure. Again, composition is usually preferable.

10.2 Virtual Functions and Polymorphism

A **virtual** function enables dynamic dispatch: the function called is determined by the actual object type at runtime, not the static type of the pointer or reference.

```
class Base {
public:
    virtual void print() const { std::cout << "Base\n"; }
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void print() const override { std::cout << "Derived\n"; }
};

void callPrint(const Base& b) {
    b.print(); // calls Base::print or Derived::print
}

int main() {
    Derived d;
    Base& ref = d;
    ref.print(); // prints "Derived"
}
```

The `override` keyword (C++11) explicitly states intent to override a base virtual function. The compiler checks that such a function exists, preventing subtle signature mismatch bugs. The `final` keyword prevents further overriding.

10.3 The Virtual Table (vtable)

Under the hood, virtual functions are implemented using a virtual table (vtable). Each class with virtual functions has a vtable: an array of function pointers. Each object contains a hidden pointer (vptr) to its class's vtable. When a virtual function is called, the compiler generates code to index into the vtable.

This adds a small overhead: one pointer per object, and an indirection per virtual call. Modern compilers can devirtualize calls when the object type is known. Do not fear virtual functions, but do not make every function virtual. Only functions that are part of the polymorphic interface should be virtual.

10.4 Virtual Destructors

Always declare a virtual destructor in a base class if objects will be deleted through a base class pointer. Without it, deleting a derived object through a base pointer invokes undefined behavior because the derived destructor is never called.

```
Base* p = new Derived();
delete p; // safe only if Base has virtual destructor
```

If a class is not meant to be a base class, make its destructor protected and non-virtual, or use `final` on the class.

10.5 Abstract Classes and Interfaces

An abstract class has at least one pure virtual function (designated by `= 0`). It cannot be instantiated. Derived classes must implement all pure virtuals to become concrete.

An interface is a class with only pure virtual functions (and a virtual destructor). C++ does not have a separate interface keyword; it uses abstract classes. This is a common pattern for defining APIs and dependency injection points.

```
class ILogger {
public:
    virtual void log(const std::string& msg) = 0;
    virtual ~ILogger() = default;
};
```

10.6 Copy Semantics

When an object is copied, a new object is created with the same state. Two special member functions control copying:

Copy constructor: `Foo(const Foo& other)` – initializes a new object as a copy. **Copy assignment operator:** `Foo& operator=(const Foo& other)` – assigns to an existing object.

If not declared, the compiler generates both, performing memberwise copy. This is correct for classes whose members are all copyable. For classes managing resources (e.g., raw pointers to heap memory), the generated copies are shallow: they copy the pointer, not the pointed-to data. This leads to double-deletes and use-after-free.

10.7 Move Semantics (Introduction)

Move semantics (C++11) transfer resources from a source object to a destination, leaving the source in a valid but unspecified state. This avoids expensive deep copies, especially for temporary objects.

Move constructor: `Foo(Foo&& other) noexcept` **Move assignment operator:** `Foo& operator=(Foo&& other) noexcept`

Move operations are triggered for rvalues (temporaries, objects marked with `std::move`). They are typically `noexcept` to enable optimizations (e.g., `std::vector` growing only moves if moves are noexcept).

```
std::string s1 = "Hello";
std::string s2 = std::move(s1); // s1's data moved to s2
// s1 is now empty (valid but unspecified)
```

10.8 The Rule of Three, Five, and Zero

Rule of Three: If a class needs a custom destructor, copy constructor, or copy assignment, it likely needs all three. This was the pre-C++11 rule.

Rule of Five (C++11): Extends Rule of Three to include move constructor and move assignment operator. If any of the five is user-defined, consider defining the others.

Rule of Zero: Design classes so that none of the five special member functions need custom implementation. Use RAII types (smart pointers, containers) as members. The compiler-generated defaults will then be correct.

```
class Good {
    std::string name;
    std::vector<int> data;
    // Compiler-generated copies, moves, and destructor are correct.
};
```

10.9 Operator Overloading

Operator overloading allows user-defined types to work with C++ operators. This can make code natural and expressive but can also be abused. Overload operators only when their meaning is obvious and intuitive.

Common overloads: `+`, `-`, `*`, `/`, `==`, `!=`, `<`, `«`, `»`, `[]`, `()`.

```
class Complex {
    double re, im;
public:
    Complex operator+(const Complex& other) const {
        return {re + other.re, im + other.im};
    }
    bool operator==(const Complex& other) const = default; // C++20
};

std::ostream& operator<<(std::ostream& os, const Complex& c) {
    return os << c.real() << " + " << c.imag() << "i";
}
```

Operators can be members or free functions. Binary operators involving a class type are often best as free functions to allow implicit conversions on both operands. C++20 introduced <=> (spaceship operator) for automatic generation of comparison operators; this will be covered in the advanced features chapter.

10.10 Composition and Aggregation

Composition models a "has-a" relationship where the contained object's lifetime is managed by the containing object. If the container is destroyed, the part is destroyed. In C++,

this is implemented with value members.

Aggregation models a "uses-a" relationship where the contained object exists independently. The container holds a pointer or reference.

Prefer composition over inheritance. Inheritance is for "is-a" relationships with polymorphic behavior. Composition is for code reuse and building complex types from simpler ones. Overuse of inheritance leads to rigid, fragile hierarchies.

Chapter 11

Namespaces and Scope Management

11.1 The Problem of Name Collisions

In large programs and when combining multiple libraries, name collisions occur when two entities have the same fully qualified name. C has flat naming; prefixing conventions (like `gtk_window_new`) are a manual workaround. C++ provides **namespaces** to organize code into logical groups and prevent collisions.

11.2 Defining Namespaces

```
namespace math {
    double sin(double x) { /* ... */ }
    const double PI = 3.1415926535;
}
```

Names can be accessed using the scope resolution operator `::`:

```
double x = math::sin(math::PI / 2);
```

Namespaces are open: multiple namespace declarations with the same name are additive. This allows separate header files to contribute to the same namespace. Namespaces can be nested.

11.3 Using Declarations and Directives

A **using declaration** brings a single name into the current scope:

```
using math::sin;
double x = sin(1.0); // no qualification needed
```

A **using directive** brings all names from a namespace into scope:

```
using namespace math; // brings everything
```

Never use using directives in header files; they pollute every translation unit that includes the header. Using directives in source files are acceptable but should be used judiciously. Prefer explicit qualification or targeted using declarations.

11.4 Anonymous Namespaces

An anonymous namespace gives its contents internal linkage, effectively hiding them from other translation units. This is the C++ replacement for `static` applied to global declarations.

```
namespace {
    void helper() { /* local to this translation unit */ }
}
```

Use anonymous namespaces for implementation details that should not be exposed. They prevent ODR violations from duplicate definitions across translation units.

11.5 Inline Namespaces (C++11)

An inline namespace makes its members appear to belong to the enclosing namespace. This is used primarily for versioning: library ABI changes can be placed in versioned inline namespaces, allowing old and new code to coexist.

```
namespace lib {
    inline namespace v2 {
        void new_feature();
    }
    namespace v1 {
        void old_feature();
    }
}
// lib::new_feature() is accessible as if in lib
```

11.6 Namespace Aliases

Long nested namespaces can be aliased:

```
namespace chrono = std::chrono;
```

This is common for deeply nested libraries.

Chapter 12

Type Casting in C++

12.1 Implicit Conversions

C++ performs implicit (automatic) conversions when an expression of one type is used where another type is expected. This includes numeric promotions (small integer types to `int`), numeric conversions (`int` to `double`), and user-defined conversions (single-argument constructors, conversion operators).

Implicit conversions can be dangerous. They may lose precision, change signedness, or trigger unintended constructors. The `explicit` keyword prevents constructors and conversion operators from being used implicitly.

```
class Foo {
public:
    explicit Foo(int x); // cannot be implicitly constructed from int
};
```

12.2 C-Style Casts

A C-style cast `(T)expr` is a blunt instrument. It tries the following sequence until one succeeds: `const_cast`, `static_cast`, `static_cast` followed by `const_cast`, `reinterpret_cast`, `reinterpret_cast` followed by `const_cast`. It can silently change meaning when code is refactored. Never use C-style casts in C++ code.

12.3 static_cast

`static_cast` performs well-defined conversions between related types: numeric conversions, upcasting in class hierarchies, downcasting (without runtime checks), and invoking explicit conversion operators. It is checked at compile time for reasonableness.

```
double d = 3.14;
int i = static\_cast<int>(d); // truncates
Base* b = static\_cast<Base*>(derived_ptr); // upcast, safe
```

12.4 dynamic_cast

`dynamic_cast` performs a runtime-checked downcast in polymorphic hierarchies. It requires the base class to have at least one virtual function. If the cast fails for a pointer, it returns `nullptr`. If the cast fails for a reference, it throws `std::bad_cast`.

```
Base* b = get_derived();
Derived* d = dynamic\_cast<Derived*>(b);
if (d) {
    d->derived_method();
}
```

`dynamic_cast` has runtime overhead and indicates a design that may be overly dependent on RTTI. Prefer virtual functions when possible. Use `dynamic_cast` for occasional type interrogation.

12.5 const_cast

`const_cast` adds or removes `const` (and `volatile`) qualifiers. It is the only cast that can remove constness. Its primary legitimate use is interfacing with legacy C APIs that lack const correctness. Using `const_cast` to modify an originally const object is undefined behavior.

```
void legacy_api(char* buffer);
const char* data = "hello";
legacy_api(const\_cast<char*>(data)); // only safe if legacy_api
   doesn't write
```

12.6 reinterpret_cast

`reinterpret_cast` reinterprets the bit pattern of an expression as a different type. It is the most dangerous cast. It does not generate any machine instructions; it just tells the compiler to treat the bits differently. Use only for low-level operations like memory-mapped I/O, serialization, and type-punning where strict aliasing rules are carefully observed.

```
int value = 0x40490FDB;
float f = reinterpret\_cast<float&>(value); // type-pun
```

Note that the above code violates strict aliasing rules and is technically undefined behavior in C++. The correct modern way is `std::bit_cast` (C++20) or `memcpy`. `reinterpret_cast` is a code smell; use it sparingly and document why.

Chapter 13

Exception Handling

13.1 The Purpose of Exceptions

Exceptions provide a mechanism for reporting and handling errors that cannot be resolved locally. Before exceptions, error handling relied on error codes, which cluttered logic, were easily ignored, and made resource cleanup difficult. Exceptions separate error handling from normal control flow.

13.2 Throwing and Catching

Throw an exception with `throw`. Catch it with a `try/catch` block.

```
try {
    if (error_condition) {
        throw std::runtime_error("Something went wrong");
    }
    // normal code
} catch (const std::runtime_error& e) {
    std::cerr << "Error: " << e.what();
} catch (const std::exception& e) {
    std::cerr << "General error: " << e.what();
} catch (...) {
    std::cerr << "Unknown error";
}
```

Exceptions are caught by type. Catch blocks are tried in order; the first matching type handles the exception. Catching by const reference avoids slicing and unnecessary copying. A `catch(...)` block catches everything.

13.3 Stack Unwinding

When an exception is thrown, the runtime unwinds the call stack: local objects are destroyed in reverse order of construction, their destructors called. This process ensures resources (memory, files, locks) are released automatically, provided they were managed by RAII objects. This is the fundamental reason RAII is critical: it makes exception safety possible.

Do not throw exceptions from destructors unless you know what you are doing. If a destructor called during stack unwinding throws, the program terminates. Destructors should be non-throwing; mark them `noexcept`.

13.4 noexcept Specifier

`noexcept` specifies that a function does not throw. The compiler can generate more efficient code for noexcept functions. In C++17, `noexcept` is part of the function type.

```
void safe_function() noexcept {
    // guarantees no exception escapes
```

```
}
```

Functions that are `noexcept` are safely used in contexts where exceptions are unacceptable (destructors, move operations, swap). Use `noexcept` for functions that logically cannot fail. The `noexcept` operator checks if an expression is noexcept: `noexcept(func())` returns `bool` at compile time.

13.5 Exception Safety Guarantees

C++ defines three levels of exception safety:

No-throw guarantee: The operation cannot throw. Destructors and move operations should aim for this.

Strong guarantee: If an exception is thrown, the program state is rolled back to what it was before the operation. Like a transaction.

Basic guarantee: If an exception is thrown, no resources leak, and objects are in a valid (if unspecified) state.

No guarantee: None of the above; exception may leave program in an inconsistent state.

Design functions to offer the strongest guarantee possible. Copy-and-swap idiom is a common technique for achieving the strong guarantee in assignment operators.

13.6 std::exception Hierarchy

The standard library provides a hierarchy of exception classes in `<stdexcept>`. All derive from `std::exception`, which has a virtual `what()` member returning a C-string description.

Key classes: `std::logic_error` (program bugs: `invalid_argument`, `out_of_range`), `std::runtime_error` (unpredictable conditions: `range_error`, `overflow_error`), `std::bad_alloc` (allocation failure), `std::bad_cast` (failed `dynamic_cast`).

Derive your own exceptions from these standard types to integrate with existing catch handlers.

Chapter 14

Move Semantics in Depth

14.1 The Problem: Unnecessary Copies

Before C++11, functions returning large objects incurred expensive deep copies. Even with optimizations like Return Value Optimization (RVO), many scenarios required copying data that was about to be destroyed. Move semantics solves this by allowing resources to be transferred cheaply from temporaries (or objects explicitly marked as movable).

14.2 lvalues, rvalues, and Value Categories

C++ expressions are classified by value category. The two primary categories are:

lvalue: An expression with an identifiable address. Variables, dereferenced pointers, array elements. lvalues can appear on the left side of assignment.

rvalue: An expression that is not an lvalue. Temporaries, literals, results of arithmetic. rvalues can appear only on the right side of assignment (traditionally).

C++11 refined this into lvalue, xvalue (eXpiring value), and prvalue (pure rvalue). lvalues and xvalues together are glvalues (generalized lvalues). Prvalues and xvalues together are rvalues. Understanding this taxonomy helps with template metaprogramming, but the practical takeaway is: an object that is about to be destroyed can be safely moved from.

14.3 Rvalue References

An rvalue reference is declared with `T&&`. Unlike lvalue references (`T&`), rvalue references bind only to rvalues. They allow us to detect when an object is temporary and can be pilfered.

```
void func(std::string&& s) {
    // s is an rvalue reference parameter
    // s itself is an lvalue here (it has a name)
}
func(getString()); // binds to temporary
std::string x;
func(x);           // error: cannot bind lvalue to rvalue ref
func(std::move(x)); // OK: std::move casts to rvalue
```

14.4 std::move

`std::move` is a cast that unconditionally converts its argument to an rvalue. It does not move anything itself; it just enables move semantics. The name is misleading. It is actually `static_cast<T&&>(x)`.

```
std::string s1 = "Hello";
std::string s2 = std::move(s1); // move constructor called
// s1 is now in a valid but unspecified state
```

After moving from an object, you can safely destroy it or assign a new value. Accessing its moved-from value (other than to query state) is usually a bug. Standard library types leave moved-from objects in an "empty" state.

14.5 Move Constructor and Move Assignment

```
class Buffer {
    char* data;
    size_t size;
public:
    // Move constructor
    Buffer(Buffer&& other) noexcept
        : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
    }

    // Move assignment
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data;          // free own resource
            data = other.data;      // steal other's resource
            size = other.size;
            other.data = nullptr;   // leave other safe
            other.size = 0;
        }
        return *this;
    }

    ~Buffer() { delete[] data; }
};
```

A moved-from object must still be destructible and assignable. Setting the source's pointer to `nullptr` prevents a double-delete. Always mark move operations `noexcept`; it enables optimizations and ensures safety guarantees.

14.6 Perfect Forwarding

Perfect forwarding allows a function template to pass arguments to another function while preserving their value category (lvalue/rvalue) and const qualification. It uses universal references and `std::forward`.

```
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

`Args&&` with template deduction is a universal (forwarding) reference: it binds to lvalues and rvalues. `std::forward` casts to an rvalue only if the original argument was an rvalue. This enables arguments to be moved into constructors when appropriate and copied otherwise, all transparently.

14.7 Universal References

A universal reference appears in a deduced context: `auto&&` or `T&&` where `T` is a template parameter being deduced. The ampersand syntax does not always mean rvalue reference. If the argument is an lvalue, `T` is deduced as `T&`; by reference collapsing, `T& &&` becomes `T&`. If the argument is an rvalue, `T` is deduced as plain `T`, giving `T&&`.

This collapsing mechanism is the key to perfect forwarding. Do not overload on universal references unless you understand the subtle interactions with other overloads; they are greedy and can hijack calls intended for other functions.

Chapter 15

Templates: Generic Programming

15.1 The Problem Templates Solve

Without templates, you would write duplicate code for different types. Overloading functions for every type is tedious and error-prone. Macros are type-unsafe and hard to debug. Templates enable writing code once that works with any type, with type checking at compile time.

15.2 Function Templates

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

int x = max(3, 5);            // T deduced as int
double y = max(3.14, 2.71); // T deduced as double
// max(3, 3.14);              // error: ambiguous deduction
```

The compiler generates a separate function for each set of template arguments used. This is called **template instantiation**. The compiler deduces template arguments from the function arguments. Deduction may fail if types are inconsistent; you can explicitly specify: `max<double>(3, 3.14)`.

15.3 Class Templates

```
template<typename T>
class Vector {
    T* data;
    size_t size;
public:
    explicit Vector(size_t n) : data(new T[n]), size(n) {}
    ~Vector() { delete[] data; }
    T& operator[](size_t i) { return data[i]; }
    size_t getSize() const { return size; }
};

Vector<int> vi(10);
Vector<std::string> vs(5);
```

Class templates are the foundation of the standard library containers. The full definition must be available in every translation unit that uses it, so class templates are typically defined entirely in headers.

15.4 Template Specialization

You can provide a specialized implementation for specific template arguments.

Full specialization: An entirely different implementation for a concrete type.

```
template<>
class Vector<bool> {
    // packed bit storage, different interface
};
```

Partial specialization: Specializing for a category of types.

```
template<typename T>
class Vector<T*> {
    // specialized behavior for pointers
};
```

15.5 Non-Type Template Parameters

Templates can accept compile-time constant values, not just types.

```
template<typename T, size_t N>
class Array {
    T data[N];
public:
    size_t size() const { return N; }
};

Array<int, 10> arr;
```

`std::array` uses this technique. Values must be compile-time constants: integral types, enums, pointers, or references with static lifetime. Since C++20, floating-point and class types with strong equality are also allowed.

15.6 Variadic Templates (C++11)

Variadic templates accept an arbitrary number of template arguments.

```
template<typename... Args>
void print(Args... args) {
    (std::cout << ... << args); // fold expression (C++17)
}
```

Template parameter packs can be expanded with `...`. Recursive unpacking is the traditional approach; fold expressions (C++17) simplify many common cases.

15.7 Fold Expressions (C++17)

Fold expressions apply a binary operator to a parameter pack:

```
template<typename... Args>
auto sum(Args... args) {
    return (args + ...); // unary right fold
}
```

There are four forms: unary left fold, unary right fold, binary left fold, binary right fold. They drastically simplify variadic template code.

15.8 Template Metaprogramming Introduction

Templates are Turing-complete at compile time. Template metaprogramming computes values and types at compile time, enabling zero-runtime-cost abstractions.

```
template<int N>
struct Factorial {
    static constexpr int value = N * Factorial<N-1>::value;
};
template<>
struct Factorial<0> {
    static constexpr int value = 1;
};

int x = Factorial<5>::value; // 120 at compile time
```

With `constexpr` functions, many metaprogramming tasks can be expressed more naturally. Still, type-level computations (type traits, SFINAE, concepts) rely heavily on template mechanisms.

15.9 SFINAE: Substitution Failure Is Not An Error

During overload resolution, if substituting a deduced template argument into a template fails, that overload is discarded rather than causing a compile error. This enables compile-time dispatch:

```
template<typename T>
auto print(T t) -> decltype(t.print(), void()) {
    t.print();
}

template<typename T>
auto print(T t) -> decltype(std::cout << t, void()) {
    std::cout << t;
}
```

SFINAE is powerful but produces inscrutable error messages. Concepts (C++20) are the modern replacement for most SFINAE use cases.

15.10 Concepts (C++20)

Concepts constrain template parameters, providing clear, compiler-checked requirements and vastly improved error messages.

```
template<typename T>
concept Printable = requires(T t) {
    { std::cout << t } -> std::same_as<std::ostream&>;
};

template<Printable T>
void output(const T& t) {
    std::cout << t;
}
```

Concepts can be used with `requires` clauses, as part of `auto` declarations, and to constrain class templates. They make generic code self-documenting and prevent misuse at the call site rather than deep in instantiation.

15.11 The Curiously Recurring Template Pattern (CRTP)

CRTP is an idiom where a class inherits from a base class template instantiated with the derived class as the template argument.

```
template<typename Derived>
class Counter {
    static int count;
public:
    Counter() { ++count; }
    ~Counter() { --count; }
    static int getCount() { return count; }
};
template<typename Derived>
int Counter<Derived>::count = 0;

class MyClass : public Counter<MyClass> {};
```

CRTP enables static polymorphism: the base class can call methods of the derived class without virtual dispatch. It is used for mixins, code injection, and compile-time method binding (e.g., `std::enable_shared_from_this`).

Chapter 16

The Standard Template Library (STL): Containers

16.1 The Philosophy of the STL

The STL was designed around three pillars: containers (data structures), algorithms (operations on data), and iterators (the glue connecting them). This orthogonal design means you can write an algorithm once and apply it to any container, and you can write a container once and have all algorithms work on it, provided you supply iterators.

The STL emphasizes value semantics, generic programming, and performance. It does not use virtual functions for polymorphism; it uses templates. Complexities are specified in the standard, allowing you to reason about algorithm performance portably.

16.2 Sequence Containers

Sequence containers store elements in linear order.

std::vector is the default container. It stores elements contiguously in dynamic memory. Random access is O(1). Insert/erase at the end is amortized O(1). Insert/erase in the middle is O(n). It doubles capacity when growing, which can cause reallocation.

```
std::vector<int> v = {1, 2, 3};
v.push_back(4);
v.pop_back();
for (const auto& x : v) { std::cout << x; }
```

std::deque (double-ended queue) provides O(1) insert/erase at both ends and O(1) random access, but elements are not stored contiguously. Use when you need fast front insertion without the overhead of a list.

std::list is a doubly-linked list. Insert/erase anywhere is O(1), but random access is O(n). Memory overhead per element (two pointers). Use when you need frequent mid-sequence insertions and deletions and are pointer-stable.

std::forward_list (C++11) is a singly-linked list with even less overhead.

16.3 Associative Containers

Associative containers maintain elements in a sorted order, implemented as balanced binary search trees (typically red-black trees).

std::set stores unique keys. **std::multiset** allows duplicates. **std::map** stores key-value pairs with unique keys. **std::multimap** allows duplicate keys.

All have O(log n) lookup, insert, and erase.

```
std::map<std::string, int> ages;
ages["Alice"] = 30;
ages.insert({"Bob", 25});
if (ages.contains("Alice")) { /* C++20 */ }
int a = ages.at("Alice"); // throws if not found
```

16.4 Unordered Associative Containers (C++11)

Implemented as hash tables, providing average O(1) lookup. Worst-case O(n) if many collisions.

std::unordered_set, **std::unordered_multiset**, **std::unordered_map**, **std::unordered_multimap**.

Custom types require a hash function (a specialization of `std::hash`) and equality operator. Use unordered containers when order is irrelevant and you need the fastest average lookup.

16.5 Container Adaptors

Adaptors provide a restricted interface on top of an underlying container.

std::stack provides LIFO (last-in-first-out) access. Defaults to `std::deque`.

std::queue provides FIFO access. Defaults to `std::deque`.

std::priority_queue provides access to the largest element. Defaults to `std::vector` with heap operations.

```
std::stack<int> s;
s.push(1); s.push(2);
int top = s.top(); // 2
s.pop();
```

16.6 std::array and std::span

std::array is a fixed-size container with zero overhead over C arrays. Size is part of the type.

std::span (C++20) is a non-owning view over contiguous data (array, vector, C array). It stores a pointer and a size. It is to contiguous containers what `string_view` is to strings.

```
void process(std::span<int> s) {
    for (int& x : s) { x *= 2; }
}
std::vector<int> v = {1, 2, 3};
process(v); // works
```

16.7 Choosing the Right Container

Container	When to Use
`std::vector`	Default choice. Contiguous storage, great cache locality.
`std::deque`	Need fast insert at front and back, no pointer stability needed.
`std::list`	Need O(1) mid-sequence insert/erase with pointer stability. Rarely the best choice.
`std::set/map`	Need ordered keys, O(log n) operations.
`std::unordered_set/map`	Need O(1) average lookup, order irrelevant.

Measure and profile before committing to a container. The theoretical complexities guide initial choice; real-world cache effects often favor contiguous containers.

Chapter 17

STL Algorithms and Iterators

17.1 The Iterator Abstraction

An iterator is an object that points to an element and can be advanced to the next. It generalizes pointers. Every STL container provides `begin()` and `end()` members. `begin()` returns an iterator to the first element; `end()` returns a sentinel one past the last element.

```
std::vector<int> v = {1, 2, 3};
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it;
}
```

Iterators decouple algorithms from containers. An algorithm operates on iterator ranges; it does not know or care what container the iterators come from.

17.2 Iterator Categories

The STL defines a hierarchy of iterator capabilities:

Input iterator: read once, one-pass. (e.g., `std::istream_iterator`) **Forward iterator:** read multiple passes. (e.g., `std::forward_list`) **Bidirectional iterator:** forward and backward. (e.g., `std::list`) **Random access iterator:** constant-time advance by N, subscripting. (e.g., `std::vector`) **Contiguous iterator (C++17):** random access with guaranteed contiguous storage.

Algorithms specify the minimum iterator category they require. Using an algorithm with a more powerful iterator may enable optimizations (e.g., `std::sort` requires random access).

17.3 Non-Mutating Algorithms

These algorithms read data without modifying it.

`std::find` locates the first element equal to a value. `std::find_if` finds the first element satisfying a predicate. `std::count` / `std::count_if` count matching elements. `std::all_of`, `std::any_of`, `std::none_of` check predicates across a range. `std::equal` compares two ranges. `std::search` finds a subsequence.

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) { std::cout << "Found at " <<
   std::distance(v.begin(), it); }
```

17.4 Mutating Algorithms

These modify elements.

`std::copy` copies one range to another. `std::move` moves elements (uses `std::move` internally). `std::transform` applies a function to each element and writes output. `std::fill` assigns a value to all elements. `std::generate` fills with results of a callable. `std::replace`

/ `std::replace_if` substitute values. `std::remove` / `std::remove_if` reorder elements to remove matches (does not resize; use erase-remove idiom).

```
std::vector<int> v = {1, 2, 3, 2, 4};
v.erase(std::remove(v.begin(), v.end(), 2), v.end());
// v is now {1, 3, 4}
```

17.5 Sorting and Searching

`std::sort` sorts in O(n log n), usually introsort (quicksort with heapsort fallback). Requires random access iterators. `std::stable_sort` preserves order of equivalent elements. `std::partial_sort` sorts only the first N elements. `std::nth_element` partitions around the Nth element.

Searching in sorted ranges: `std::binary_search`, `std::lower_bound`, `std::upper_bound`, `std::equal_range`.

```
std::vector<int> v = {5, 2, 8, 1, 3};
std::sort(v.begin(), v.end());
bool found = std::binary_search(v.begin(), v.end(), 3);
```

17.6 Heap Algorithms

The STL provides heap operations on random access ranges: `std::make_heap`, `std::push_heap`, `std::pop_heap`, `std::sort_heap`. These underlie `std::priority_queue`.

17.7 Lambda Expressions and Algorithms

Algorithms come alive with lambdas. Custom comparators, predicates, and transformations are expressed inline.

```
std::vector<int> v = {1, -2, 3, -4};
std::sort(v.begin(), v.end(), [](int a, int b) {
    return std::abs(a) < std::abs(b);
});
// v sorted by absolute value
```

17.8 Parallel Algorithms (C++17)

Many algorithms accept an execution policy as the first argument: `std::execution::seq` (sequential), `std::execution::par` (parallel), `std::execution::par_unseq` (parallel and vectorized).

```
#include <execution>
std::sort(std::execution::par, v.begin(), v.end());
```

Using parallel policies requires care to avoid data races and ensure thread safety. Not all implementations may support all policies; check the compiler documentation.

Chapter 18

Advanced Standard Library Types

18.1 std::pair and std::tuple

`std::pair` stores two values, accessible via `.first` and `.second`. `std::make_pair` deduces types. Structured bindings (C++17) unpack pairs:

```
std::pair<std::string, int> p{"Alice", 30};
auto [name, age] = p;
```

`std::tuple` generalizes to any number of values. `std::make_tuple`, `std::get<N>`, `std::tie`:

```
std::tuple<int, double, char> t{1, 3.14, 'a'};
int i = std::get<0>(t);
auto [x, y, z] = t; // structured bindings
```

18.2 std::optional (C++17)

`std::optional<T>` represents a value that may or may not be present. It replaces sentinel values and nullable pointers for "no result" semantics.

```
std::optional<int> divide(int a, int b) {
    if (b == 0) return std::nullopt;
    return a / b;
}

if (auto result = divide(10, 2)) {
    std::cout << *result;
}
// or: std::cout << result.value_or(-1);
```

`value()` throws `std::bad_optional_access` if no value. `value_or(default)` provides a fallback. `has_value()` checks.

18.3 std::variant (C++17)

`std::variant<Ts...>` is a type-safe union. It holds a value of exactly one of the specified types at a time.

```
std::variant<int, double, std::string> v;
v = 42;
v = "hello";

std::visit([](auto&& arg) {
    std::cout << arg;
}, v);
```

`std::visit` applies a visitor to the active value. `std::get<T>` retrieves by type; `std::get_if<T>` returns a pointer or null.

18.4 std::any (C++17)

`std::any` can hold a value of any copy-constructible type. It performs type erasure.

```
std::any a = 42;
a = std::string("hello");
std::cout << std::any\_cast<std::string>(a);
```

`any_cast` throws `std::bad_any_cast` on type mismatch. `std::any` is useful for truly type-agnostic storage, but prefer `std::variant` when the set of possible types is known.

18.5 std::span (C++20)

`std::span<T>` is a non-owning view over a contiguous sequence. It consists of a pointer and a size. It works with arrays, vectors, and C arrays seamlessly.

```
void sum(std::span<const int> data) {
    int total = 0;
    for (int x : data) total += x;
}

std::vector<int> v = {1, 2, 3, 4};
sum(v); // OK
int arr[] = {5, 6, 7};
sum(arr); // OK
```

18.6 Ranges and Views (C++20)

`<ranges>` introduces a new paradigm for composing algorithms. A **range** is anything with `begin()` and `end()`. A **view** is a lightweight, lazy-evaluated range.

```
#include <ranges>
std::vector<int> v = {1, 2, 3, 4, 5, 6};
auto even = v | std::views::filter([](int x) { return x % 2 == 0; })
              | std::views::transform([](int x) { return x * x; });
for (int x : even) { std::cout << x << ' '; } // 4 16 36
```

Views are composable, lazy, and non-owning. They evaluate only as the range is iterated. This enables functional-style programming with zero overhead. **Adaptors** like `filter`, `transform`, `take`, `drop`, `reverse`, and `join` create views.

18.7 Chrono Library

`<chrono>` provides time utilities: durations, time points, and clocks.

```
#include <chrono>
using namespace std::chrono;

auto start = steady\_clock::now();
// ... work ...
auto end = steady\_clock::now();
auto elapsed = duration\_cast<milliseconds>(end - start);
std::cout << elapsed.count() << "ms\n";
```

`std::chrono::steady_clock` is monotonic and suitable for measuring intervals. `system_clock` represents wall-clock time. `high_resolution_clock` is the most precise clock available. C++20 added calendar and time zone support.

18.8 Filesystem Library (C++17)

`<filesystem>` provides portable file and directory manipulation.

```
#include <filesystem>
namespace fs = std::filesystem;

fs::path p = "data/config.txt";
if (fs::exists(p)) {
    auto size = fs::file\_size(p);
    std::cout << "Size: " << size;
}

for (const auto& entry : fs::directory\_iterator(".")) {
    std::cout << entry.path() << '\n';
}
```

The library handles path separators, symlinks, permissions, and recursive directory traversal. It replaces platform-specific APIs and C file functions for directory operations.

18.9 Regular Expressions

`<regex>` provides regex matching, searching, and replacement.

```
#include <regex>
std::string text = "Email me at alice@example.com";
std::regex pattern(R"((\w+)@(\w+\.\w+))");
std::smatch match;
if (std::regex_search(text, match, pattern)) {
    std::cout << "User: " << match[1] << '\n';
    std::cout << "Domain: " << match[2] << '\n';
}
```

The C++ regex library has performance limitations on some implementations. For simple pattern matching, manual string functions may be faster. For heavy regex work, consider specialized libraries like RE2.

18.10 Random Number Generation

`<random>` provides a modern random number framework, replacing `rand()` which has poor quality and global state.

```
#include <random>
std::mt19937 engine{std::random_device{}()}; // Mersenne Twister
std::uniform_int_distribution<int> dist(1, 6);
int roll = dist(engine);
```

Engines generate raw random bits. Distributions shape them into desired probability distributions. `std::random_device` provides non-deterministic random numbers (if hardware supports it) and is ideal for seeding engines.

18.11 std::bitset

`std::bitset<N>` provides a fixed-size sequence of bits with convenient operations: set, reset, flip, test, count, and bitwise operators.

```
std::bitset<8> flags;
flags.set(3);        // set bit 3
flags[5] = true;     // set bit 5
std::cout << flags.count(); // number of set bits
std::cout << flags.to_string(); // binary representation
```

Chapter 19

Input/Output Streams and File Handling

19.1 The Stream Abstraction

C++ I/O is based on streams: a sequence of bytes flowing from a source to a destination. The `<ios>`, `<istream>`, `<ostream>`, and `<iostream>` headers provide the class hierarchy. `std::cin` is an `istream`, `std::cout` and `std::cerr` are `ostreams`.

The « (insertion) and » (extraction) operators are overloaded for fundamental types. Streams manage formatting state (width, precision, base) through manipulators.

19.2 Formatted I/O

```
#include <iomanip>
double pi = 3.1415926535;
std::cout << std::fixed << std::setprecision(3) << pi; // "3.142"
std::cout << std::hex << std::uppercase << 255;          // "FF"
```

Manipulators like `std::hex`, `std::scientific`, `std::setw`, `std::setfill` control formatting. They are sticky, affecting subsequent output until changed.

19.3 String Streams

`<sstream>` provides `std::stringstream` (input/output), `std::istringstream`, and `std::ostringstream`. They allow treating strings as streams, useful for parsing and formatting.

```
std::ostringstream oss;
oss << "Value: " << 42;
std::string result = oss.str();

std::istringstream iss("123 456");
int a, b;
iss >> a >> b;
```

19.4 File Streams

`<fstream>` provides `std::ifstream` (input), `std::ofstream` (output), and `std::fstream` (both).

```
#include <fstream>
std::ofstream out("output.txt");
if (!out) { /* error */ }
out << "Hello, file!" << std::endl;
out.close(); // optional; destructor closes

std::ifstream in("input.txt");
std::string line;
while (std::getline(in, line)) {
    std::cout << line << '\n';
```

```
}
```

File streams use RAII: the destructor closes the file. Open modes (`std::ios::in`, `std::ios::out`, `std::ios::app`, `std::ios::binary`) control behavior. Binary mode prevents newline translation on Windows.

19.5 Error Handling in Streams

Streams maintain state flags: `good()`, `eof()`, `fail()`, `bad()`. After an operation, check the stream state:

```
int x;
if (std::cin >> x) {
    // success
} else {
    std::cin.clear(); // reset error state
    std::cin.ignore(std::numeric\_limits<std::streamsize>::max(),
        '\n');
}
```

Failing to clear the error state after a failed operation will cause all subsequent I/O on that stream to silently fail.

19.6 C++20 Format Library

`<format>` (C++20) provides `std::format`, a type-safe alternative to `printf` and stream I/O for string formatting:

```
#include <format>
std::string s = std::format("Hello, {}! The answer is {}.", "World",
    42);
// "Hello, World! The answer is 42."
```

`std::format` uses Python-like format strings, is locale-independent by default, and is significantly faster than stringstreams. `std::print` (C++23) combines formatting with output.

Chapter 20

Concurrency and Parallelism

20.1 The Threading Model

C++11 introduced a standardized threading library. Prior to that, threading required platform-specific APIs (pthreads on POSIX, Win32 threads on Windows). The standard library now provides portable threads, mutual exclusion, condition variables, and atomics.

A **thread** is an independent execution context with its own stack but shared address space. The C++ memory model guarantees that operations are ordered in some interleaving of the threads' operations, but without synchronization, the order is unpredictable.

20.2 std::thread

```
#include <thread>
void worker(int id) {
    std::cout << "Thread " << id << " working\n";
}

std::thread t1(worker, 1);
std::thread t2(worker, 2);

t1.join(); // wait for t1 to finish
t2.join(); // wait for t2 to finish
```

`std::thread` represents a thread of execution. Construct it with a callable and arguments. Call `join()` to wait for completion or `detach()` to let it run independently (the program must ensure detached threads complete before `main` exits). If `std::thread` is destroyed without being joined or detached, the program terminates.

20.3 Mutexes and Locks

A **mutex** (mutual exclusion) protects shared data from concurrent access.

```
#include <mutex>
std::mutex mtx;
int shared_data = 0;

void increment() {
    std::lock\_guard<std::mutex> lock(mtx);
    ++shared\_data;
} // lock released automatically when lock\_guard destroyed
```

`std::lock_guard` is the simplest RAII lock: it locks the mutex on construction and unlocks on destruction. `std::unique_lock` provides more flexibility: deferred locking, timed locking, and explicit unlock/lock. `std::scoped_lock` (C++17) can lock multiple mutexes simultaneously, avoiding deadlock.

```
std::mutex mtx1, mtx2;
std::scoped_lock lock(mtx1, mtx2); // locks both atomically
```

20.4 Deadlock and How to Avoid It

Deadlock occurs when two threads each hold a lock the other needs. Four conditions are necessary: mutual exclusion, hold and wait, no preemption, and circular wait. Eliminate any one to prevent deadlock.

Strategies:

- Always acquire locks in a consistent global order.
- Use `std::lock` or `std::scoped_lock` for multiple mutexes.
- Avoid nested locks where possible.
- Use lock-free data structures.

20.5 Condition Variables

Condition variables allow threads to wait until a certain condition becomes true, avoiding busy-waiting.

```
std::mutex mtx;
std::condition\_variable cv;
bool ready = false;
int data;

// Producer
void produce() {
    {
        std::lock_guard lock(mtx);
        data = compute_data();
        ready = true;
    }
    cv.notify_one();
}

// Consumer
void consume() {
    std::unique\_lock lock(mtx);
    cv.wait(lock, []{ return ready; }); // unlocks while waiting
    use_data(data);
}
```

The predicate in `wait` prevents spurious wakeups. Always wait with a condition. `notify_one` wakes one waiting thread; `notify_all` wakes all.

20.6 Futures, Promises, and async

Futures represent a value that will be available later. **Promises** set that value. `std::async` launches a task asynchronously and returns a future.

```
#include <future>
int compute() { return 42; }

std::future<int> result = std::async(std::launch::async, compute);
// ... do other work ...
int value = result.get(); // blocks if not ready
```

`std::packaged_task` wraps a callable into a future/promise pair. `std::shared_future` allows multiple consumers. `std::future::wait_for` and `wait_until` enable timeouts.

20.7 Atomics

Atomic operations are indivisible; they cannot be interrupted by other threads. `<atomic>` provides `std::atomic<T>` for lock-free, thread-safe operations on shared variables without mutexes.

```
std::atomic<int> counter{0};
counter.fetch_add(1);
int old = counter.exchange(0);
```

Atomic types support load, store, exchange, compare-and-swap (`compare_exchange_weak`, `compare_exchange_strong`), and fetch-and-op. They are the building blocks for lock-free data structures.

20.8 Memory Ordering

Atomic operations accept a `std::memory_order` parameter. The default is `memory_order_seq_cst` (sequentially consistent), which provides the strongest guarantees and the most overhead. Relaxed orderings (`memory_order_relaxed`, `acquire`, `release`, `acq_rel`) allow the compiler and hardware to reorder memory operations more aggressively, potentially improving performance on weak memory model architectures.

```
atomic\_bool flag{false};
// Writer
flag.store(true, std::memory\_order\_release);
// Reader
if (flag.load(std::memory\_order\_acquire)) {
    // guaranteed to see writes before the store
}
```

Memory ordering is complex. Default to sequential consistency unless profiling shows a bottleneck. Incorrect ordering leads to elusive, catastrophic bugs.

Chapter 21

Smart Pointers and RAII

21.1 The Tragedy of Raw Pointers

Raw pointers (`T*`) are powerful but dangerous. They do not encode ownership. Is the pointer owning (must be deleted) or non-owning? Who is responsible for deletion? What if an exception is thrown before `delete`? These questions lead to memory leaks, double-deletes, and use-after-free.

RAII (Resource Acquisition Is Initialization) is the C++ answer: bind resource lifetime to object lifetime. Smart pointers apply RAII to dynamic memory.

21.2 std::unique_ptr

`std::unique_ptr` owns a dynamically allocated object exclusively. It cannot be copied; only moved. When the `unique_ptr` is destroyed, it deletes the owned object.

```
#include <memory>
auto ptr = std::make\_unique<Widget>(arg1, arg2); // preferred
    creation
// ptr owns the Widget
auto ptr2 = std::move(ptr); // ptr2 now owns; ptr is nullptr
```

`std::make_unique` (C++14) allocates and constructs in one step, avoiding a raw `new`. Always prefer it. `unique_ptr` is zero-overhead in size and performance compared to a raw pointer. It can store custom deleters for non-`delete` cleanup (file handles, etc.).

21.3 std::shared_ptr

`std::shared_ptr` enables shared ownership. Multiple `shared_ptrs` can point to the same object. An internal reference count tracks them. When the last `shared_ptr` is destroyed, the object is deleted.

```
auto sp1 = std::make\_shared<Widget>();
{
    auto sp2 = sp1; // ref count now 2
} // sp2 destroyed, ref count back to 1
// sp1 still valid
```

`std::make_shared` allocates the object and the control block (containing ref counts) in a single memory block, improving cache locality and reducing allocation overhead. `shared_ptr` is heavier than `unique_ptr`: it requires atomic ref count operations and a control block.

21.4 std::weak_ptr

`std::weak_ptr` observes an object owned by `shared_ptr` without contributing to the reference count. It can be promoted to a `shared_ptr` using `lock()`, which returns null if the object has been destroyed.

```
std::weak\_ptr<Widget> observer = sp1;
if (auto sp = observer.lock()) {
    sp->use();
} else {
    std::cout << "Object gone";
}
```

`weak_ptr` breaks cyclic references. Two objects holding `shared_ptrs` to each other will never be destroyed; using `weak_ptr` in one direction resolves the cycle.

21.5 Choosing the Right Smart Pointer

Default: `std::unique_ptr`. It expresses exclusive ownership clearly, has zero overhead, and can always be converted to `shared_ptr` if sharing becomes necessary.

Use `std::shared_ptr` only when ownership is truly shared. Shared ownership is often a design smell: it can indicate unclear ownership semantics. Consider whether a single owner with observers might work.

Use `std::weak_ptr` to break cycles or to cache optional references without extending lifetime.

Never use raw pointers for ownership. Raw pointers are for non-owning observation (e.g., function parameters, iterators). Use `observer_ptr<T>` (not yet standardized but common in guidelines) for explicit non-owning semantics.

21.6 RAII Beyond Memory

RAII applies to any resource: file handles, locks, sockets, database connections. Wrapping acquisition and release in an object's constructor/destructor guarantees proper cleanup regardless of how a scope is exited.

```
class ScopedFile {
    FILE* file;
public:
    ScopedFile(const char* path) : file(fopen(path, "r")) {}
    ~ScopedFile() { if (file) fclose(file); }
    // non-copyable, movable
};
```

The standard library uses RAII extensively: `std::fstream` for files, `std::lock_guard` for mutexes, smart pointers for memory.

Chapter 22

Modern C++ Features and Best Practices

22.1 Modules (C++20)

Modules replace the textual `#include` model with a structured, compiler-understood import system. They dramatically improve compile times, eliminate macro leakage, and provide better encapsulation.

```
// my_module.ixx (module interface unit)
export module my\_module;

export int add(int a, int b) { return a + b; }

// main.cpp
import my\_module;
int main() {
    return add(2, 3);
}
```

Modules are a fundamental shift in how C++ programs are structured. Adoption is ongoing; build system support (CMake 3.28+) and compiler maturity vary. For new projects targeting C++20, modules are the future.

22.2 Undefined Behavior

Undefined behavior (UB) occurs when the program violates a language rule for which the standard imposes no required behavior. The compiler may assume UB never happens and optimize accordingly, often with surprising results.

Common UB sources:

- Dereferencing null or dangling pointers
- Out-of-bounds array access
- Signed integer overflow
- Data races
- Using a variable before initialization
- Violating strict aliasing rules

UB is not a runtime error; the program may appear to work on one platform and fail catastrophically on another. Tools like AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) detect many UB categories. Compile with `-fsanitize=address,undefined` during development.

22.3 Performance Considerations

Modern C++ allows writing fast code by understanding how hardware works:

Cache friendliness: Access memory sequentially. `std::vector` is cache-friendly; `std::list` is not. Data-oriented design organizes data for access patterns.

Branch prediction: CPUs speculate on branch outcomes. Predictable branches are fast; unpredictable ones cause pipeline stalls.

Inlining: Function call overhead is negligible for large functions but matters for trivial accessors. Mark small, frequently called functions `inline` (or define them in the class body). Trust the compiler; profile before micro-optimizing.

Copy elision: Since C++17, return value optimization (RVO) is mandatory in certain cases. The compiler constructs the return value directly in the caller's context, eliminating copies and moves.

```
Widget create() {
    return Widget{}; // guaranteed elision in C++17
}
```

22.4 CMake Fundamentals

CMake is the de facto build system generator for C++. A minimal `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.15)
project(MyProject LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(my_app main.cpp helper.cpp)
```

Modern CMake (3.0+) uses targets with properties (`target_include_directories`, `target_link_librari` `target_compile_features`). Avoid global settings; target-scoped commands are composable and less error-prone.

Static library:

```
add_library(mylib STATIC src1.cpp src2.cpp)
target\_link\_libraries(my_app PRIVATE mylib)
```

Shared library:

```
add_library(mylib SHARED src1.cpp)
```

22.5 Static and Shared Libraries

A **static library** (.a on Unix, .lib on Windows) is an archive of object files linked into the executable at build time. The resulting binary is self-contained and larger.

A **shared library** (.so, .dll) is loaded at runtime. Multiple executables can share one copy in memory. Updates to the library do not require recompilation of clients, but API/ABI compatibility must be maintained.

ABI (Application Binary Interface) is the runtime contract: calling conventions, name mangling, object layout. ABI is platform- and compiler-specific. C++ has no standard ABI; mixing compilers or compiler versions can cause link errors or runtime crashes. Use `extern "C"` to export C-compatible interfaces from shared libraries.

22.6 Naming Conventions

Consistency is more important than the specific convention. Common C++ styles:

Classes/Structs: PascalCase (`WidgetFactory`) **Functions/Methods:** camelCase or snake_case (`getName`, `get_name`) **Variables:** snake_case (`employee_count`) **Constants:** kPascalCase or ALL_CAPS (`kMaxSize`) **Member variables:** trailing underscore (`name_`) or m_ prefix (`m_name`) **Namespaces:** snake_case, usually one word (`std`) **Macros:** ALL_CAPS only; prefer inline functions and constexpr

22.7 Common Interview Mistakes

Knowing these pitfalls helps in coding interviews and in real code:

- Writing `using namespace std;` in a header.
- Forgetting virtual destructors in polymorphic bases.
- Passing large objects by value instead of const reference.
- Using `new` and `delete` instead of smart pointers.
- Ignoring the Rule of Three/Five/Zero.
- Dereferencing an iterator after modifying the container (invalidation).
- Returning a reference or pointer to a local variable.
- Assuming `std::move` moves something (it is a cast).
- Confusing `std::remove` (algorithm) with container's `erase`.
- Forgetting to check `std::cin` success after input.
- Writing `catch(...)` without re-throwing in destructor contexts.
- Assuming `int` is 4 bytes on all platforms.

22.8 Debugging and Profiling

Debuggers: GDB (command-line, Linux), LLDB (macOS), Visual Studio Debugger (Windows). Learn to set breakpoints, step through code, inspect variables, and examine call stacks. Debug builds (`-g`) include symbol information.

Sanitizers: Compile with `-fsanitize=address` (detects memory errors), `-fsanitize=undefined` (detects UB), `-fsanitize=thread` (detects data races). They have runtime overhead but catch elusive bugs.

Profilers: perf (Linux), Instruments (macOS), Visual Studio Profiler. Profile before optimizing. Measure CPU time, cache misses, allocations. Common tools like Valgrind (memcheck, callgrind) and gperftools provide detailed insights.

Static Analysis: clang-tidy, Cppcheck, and compiler warnings (`-Wall -Wextra -Wpedantic`) catch potential issues before runtime. Treat warnings as errors (`-Werror`) to enforce clean code.

Chapter 23

C++23 and Beyond

23.1 Key C++23 Features

C++23 is a smaller increment compared to C++20 but adds significant quality-of-life improvements:

std::expected<T, E>: A type for error handling that holds either a value or an error, similar to Rust's `Result`. It provides monadic operations (`and_then`, `or_else`, `transform`).

std::mdspan: A multidimensional span view over contiguous data. Enables elegant and efficient linear algebra operations.

std::print / std::println: Direct formatted output to the console without the ceremony of streams.

std::flat_map / std::flat_set: Sorted associative containers backed by contiguous storage, providing cache-friendliness and lower memory overhead for small to medium datasets.

Explicit object parameters (`this` parameter): Allows deducing the object type, simplifies certain CRTP patterns and enables self-returning member functions without redundant overloads.

Monadic operations for `std::optional`: `and_then`, `or_else`, `transform` allow chaining optional operations without nested if-checks.

23.2 The Future of C++

C++ continues to evolve toward greater safety, expressiveness, and performance. Core guidelines and lifetime analysis aim to eliminate common safety pitfalls. Reflection, pattern matching, and contracts are proposed for future standards. C++ remains the language of choice for performance-critical systems while becoming increasingly accessible.

The journey from "C with Classes" to modern C++ has been long but coherent. The zero-overhead principle, the commitment to backward compatibility, and the steady expansion of the standard library have made C++ a versatile tool that rewards deep understanding. The concepts in this handbook form the foundation for a lifetime of effective C++ programming.